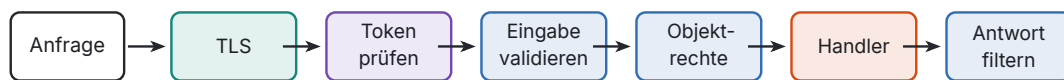


Secure API Design

REST-APIs mit FastAPI absichern: OIDC mit Authentik, Autorisierung, Validierung und die OWASP API Security Top 10



jede Stufe kann ablehnen: Verteidigung in der Tiefe

Inhalt

Bevor es losgeht	3	8.3 CORS richtig verstehen	24
Aufbau	3	8.4 Cookie-Sessions und CSRF	24
Konventionen	3	8.5 Konfiguration und Geheimnisse	24
TEIL I · GRUNDLAGEN		8.6 Dokumentation und Debug in Produktion	25
1 Ein Bedrohungsmodell für APIs	4	9 SSRF und fremde APIs	26
1.1 Vertrauensgrenzen	4	9.1 Server Side Request Forgery	26
1.2 Die OWASP API Security Top 10	4	9.2 Fremde APIs konsumieren	27
1.3 Grundprinzipien	5	9.3 Webhooks signieren und Replays verhindern	27
2 Authentifizierung mit OAuth 2.0, OIDC und Authentik	6	TEIL III · BETRIEB	
2.1 Die Rollen	6	10 Inventar, Versionen und Lebenszyklus	29
2.2 Der Authorization Code Flow mit PKCE	6	10.1 Die OpenAPI-Beschreibung als Inventar	29
2.3 Maschine zu Maschine: Client Credentials	7	10.2 Versionieren und abkündigen	29
2.4 Authentik einrichten	7	10.3 Umgebungen	30
2.5 Access-Token, ID-Token, Refresh-Token	8	10.4 Verantwortliches API-Inventar	30
3 Tokens in FastAPI prüfen	9	11 Sicherheit automatisiert testen	31
3.1 Der Prüfvertrag	9	11.1 Statische Analyse und Abhängigkeiten	31
3.2 Lokale JWT-Prüfung	10	11.2 Rechte als Tabelle testen	31
3.3 Scopes und Gruppen prüfen	11	11.3 Fuzzing aus der OpenAPI-Beschreibung	32
3.4 Widerruf und gebundene Tokens	12	11.4 Dynamischer Scan mit OWASP ZAP	32
TEIL II · DIE RISIKEN IN DER PRAXIS		11.5 In der CI	32
4 Autorisierung: Objekte und Funktionen	13	ANHANG	
4.1 BOLA: Broken Object Level Authorization	13	A Checkliste für API-Reviews	33
4.2 Autorisierung auf Funktionsebene	14	B Glossar	34
4.3 Autorisierung testen	14	Literaturverzeichnis	36
5 Datenmodelle: was hinein und was hinaus darf	16		
5.1 Ein Modell pro Richtung	16		
5.2 Eingaben begrenzen und validieren	17		
5.3 Datei-Uploads	17		
6 Ressourcen und Geschäftsabläufe schützen	18		
6.1 Grenzen auf allen Ebenen	18		
6.2 Rate Limiting	18		
6.3 Idempotenz für kritische Abläufe	19		
6.4 Missbrauch legitimer Abläufe	20		
7 Fehler, Logging und Datensparsamkeit	21		
7.1 Einheitliche Fehler nach RFC 9457	21		
7.2 Eine Request-ID für jede Anfrage	22		
7.3 Was ins Log gehört und was nicht	22		
8 Transport, Header, CORS und Konfiguration	23		
8.1 TLS überall	23		
8.2 Sicherheits-Header für APIs	23		

Bevor es losgeht

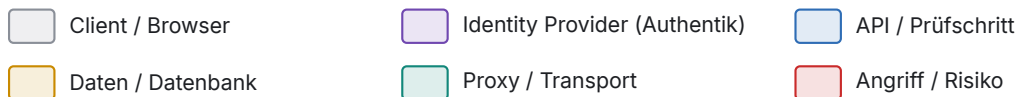
Fast jede Anwendung bietet heute eine API an, und viele Angriffe auf Webanwendungen treffen genau diese Schnittstelle. Die Fehler sind dabei selten exotisch: Eine API liefert Daten eines fremden Kontos aus, weil nur geprüft wurde, **ob** jemand angemeldet ist, aber nicht, **wem** das Objekt gehört [1]. Ein Endpunkt übernimmt ein Feld `is_admin` aus der Anfrage. Ein Token wird akzeptiert, das für eine ganz andere Anwendung ausgestellt wurde. Dieses Handbuch zeigt, wie man solche Fehler systematisch vermeidet, am Beispiel von FastAPI.

Aufbau

- **Teil I, Grundlagen:** ein Bedrohungsmodell für APIs, Authentifizierung mit OAuth 2.0 und OpenID Connect über Authentik und die korrekte Prüfung von Tokens in FastAPI.
- **Teil II, die Risiken in der Praxis:** entlang der OWASP API Security Top 10 (Ausgabe 2023) [2] Autorisierung, Datenmodelle, Ressourcenverbrauch, Fehlerbehandlung und Logging, Transport und Konfiguration sowie SSRF und fremde APIs.
- **Teil III, Betrieb:** Versionierung und Inventar sowie automatisierte Sicherheitstests in der CI.
- **Anhang:** eine Checkliste für Reviews und ein Glossar.

Konventionen

Code und Ausgaben erscheinen wie in den Handbüchern zu Git und Docker. Codebeispiele mit Sicherheitslücken sind ausdrücklich markiert (**unsicher**), die korrigierte Fassung folgt direkt darunter. In den Grafiken gilt:



Das durchgängige Beispiel ist die `notizen`-API aus dem Docker-Handbuch: FastAPI mit PostgreSQL, bei der jeder Benutzer eigene Notizen anlegt, liest und teilt, dazu Administratoren mit erweiterten Rechten und ein Hintergrunddienst, der sich ohne Benutzer an der API anmeldet. Als Identity Provider dient **Authentik** unter `auth.example.com`, die API läuft unter `api.example.com`.

MERKE

Sicherheit einer API entsteht nicht durch ein einzelnes Werkzeug, sondern durch viele einfache Prüfungen, die **jede** Anfrage durchläuft. Das Leitmotiv dieses Buchs: **Nichts, was vom Client kommt, ist vertrauenswürdig**, weder Parameter noch Felder noch Header, und jede Prüfung muss im Zweifel ablehnen.

1 Ein Bedrohungsmodell für APIs

Eine Webanwendung mit Oberfläche verbirgt einiges hinter Formularen und Schaltflächen. Eine API dagegen legt ihre Funktionen offen: Jeder Endpunkt, jeder Parameter und jedes Feld lässt sich mit `curl` direkt ansprechen, in beliebiger Reihenfolge, mit beliebigen Werten und in beliebiger Menge. Die Oberfläche (Web-App, Mobile-App) ist nur **ein** möglicher Client. Ein Angreifer schreibt sich einfach seinen eigenen.

1.1 Vertrauensgrenzen

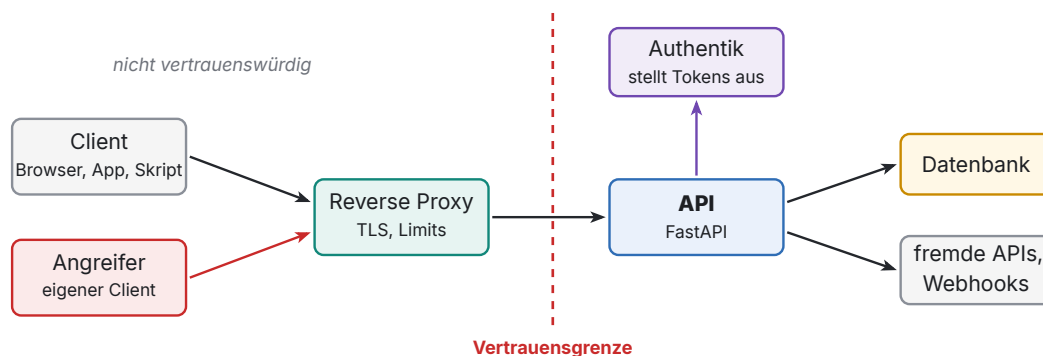


Abb. 1: Alles links der Grenze steht unter Kontrolle des Clients, also potenziell des Angreifers.

An jeder Vertrauensgrenze muss geprüft werden, was hereinkommt. Für eine API bedeutet das: Jede Anfrage wird so behandelt, als käme sie von einem Angreifer, der die API genau kennt. Dass die eigene Web-App nur gültige IDs und nur erlaubte Felder schickt, ist ohne Bedeutung, denn niemand zwingt einen Angreifer, die Web-App zu benutzen. Auch Antworten fremder APIs und Daten aus Webhooks liegen außerhalb der eigenen Kontrolle.

1.2 Die OWASP API Security Top 10

Das Open Worldwide Application Security Project (OWASP) veröffentlicht eine Liste der häufigsten Sicherheitsrisiken speziell für APIs. Die aktuelle Ausgabe von 2023 [2] bildet das Gerüst für Teil II:

Nr.	Risiko	Kapitel
API1	Broken Object Level Authorization (BOLA): Zugriff auf fremde Objekte über deren ID	4
API2	Broken Authentication: fehlerhafte Anmeldung und Token-Prüfung	2, 3
API3	Broken Object Property Level Authorization: zu viele Felder lesbar oder schreibbar (Mass Assignment)	5
API4	Unrestricted Resource Consumption: keine Grenzen für Menge, Größe, Häufigkeit	6
API5	Broken Function Level Authorization: Admin-Funktionen für normale Benutzer erreichbar	4
API6	Unrestricted Access to Sensitive Business Flows: automatisierter Missbrauch legitimer Abläufe	6
API7	Server Side Request Forgery (SSRF): die API ruft vom Angreifer gewählte URLs auf	9
API8	Security Misconfiguration: Header, CORS, Debug, Fehlermeldungen	7, 8
API9	Improper Inventory Management: vergessene alte Versionen und Endpunkte	10
API10	Unsafe Consumption of APIs: Daten fremder APIs ungeprüft übernehmen	9

Auffällig ist, dass drei der ersten fünf Punkte Autorisierung betreffen [2]. Die meisten API-Lücken sind keine technischen Schwachstellen im engeren Sinn, sondern fehlende Prüfungen in der Geschäftslogik. Kein Scanner und keine Firewall findet sie zuverlässig, sie müssen im Code vermieden werden.

1.3 Grundprinzipien

Prinzip	Bedeutung für eine API
Standardmäßig ablehnen	Jeder Endpunkt ist geschützt, außer er wird ausdrücklich öffentlich gemacht. Eine vergessene Prüfung führt dann zu einem Fehler, nicht zu einer Lücke.
Minimale Rechte	Tokens enthalten nur die Scopes, die ein Client braucht. Dienste haben eigene Konten mit eigenen Rechten.
Verteidigung in der Tiefe	Proxy, Framework, Geschäftslogik und Datenbank prüfen jeweils selbst. Fällt eine Schicht aus, hält die nächste.
Im Fehlerfall schließen	Kann eine Prüfung nicht durchgeführt werden (etwa weil die Schlüssel des Identity Providers nicht abrufbar sind), wird die Anfrage abgelehnt, nicht durchgelassen.
Datensparsamkeit	Was die API nicht speichert und nicht ausliefert, kann auch nicht abfließen.
Explizit statt implizit	Eingabe- und Ausgabemodelle legen genau fest, welche Felder erlaubt sind, statt alles durchzureichen.

2 Authentifizierung mit OAuth 2.0, OIDC und Authentik

Selbst gebaute Anmeldungen mit Passwort-Tabellen, Sessions und Passwort-Reset sind eine der größten Fehlerquellen [3]. Moderne APIs delegieren die Anmeldung an einen **Identity Provider** (IdP), der Benutzer verwaltet, Zwei-Faktor-Authentifizierung erzwingt und signierte Tokens ausstellt. Die API selbst prüft nur noch diese Tokens. In diesem Buch übernimmt **Authentik** diese Rolle, ein quelloffener IdP, der selbst gehostet wird [4].

2.1 Die Rollen

Rolle (OAuth-Begriff) [5]	Im Beispiel	Aufgabe
Resource Owner	Benutzerin	besitzt die Daten und erteilt Zugriff
Client	Web-App, Hintergrunddienst	möchte im Namen der Benutzerin oder im eigenen Namen auf die API zugreifen
Authorization Server	Authentik	authentifiziert, fragt ggf. nach Zustimmung und stellt Tokens aus
Resource Server	notizen-API	nimmt Tokens entgegen, prüft sie und liefert Daten

OAuth 2.0 regelt, wie ein Client ein Zugriffstoken für eine API bekommt (Autorisierung) [5]. **OpenID Connect** (OIDC) setzt darauf auf und liefert zusätzlich Informationen darüber, wer sich angemeldet hat (Authentifizierung), unter anderem im *ID-Token* [6].

2.2 Der Authorization Code Flow mit PKCE

Für Web-, Mobile- und Desktop-Apps mit geeignetem Browser ist der *Authorization Code Flow* mit *PKCE* (§256) der empfohlene Ablauf [7], [8]. Geräte mit eingeschränkter Eingabe und ohne geeigneten Browser verwenden den *Device Authorization Grant* [9]. Die früher verbreiteten Varianten *Implicit* (Token direkt in der URL) und *Resource Owner Password* (Client sammelt das Passwort ein) gelten als unsicher [7] und sollten in Authentik gar nicht erst aktiviert werden [10].

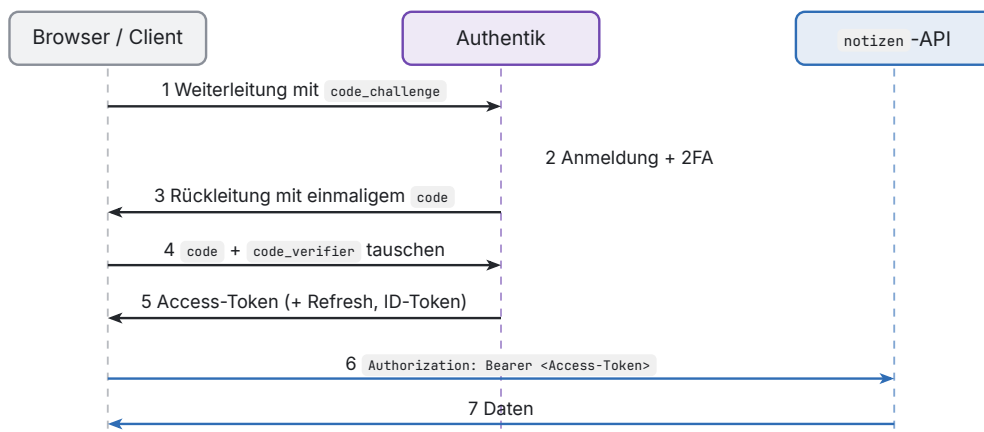


Abb. 2: Authorization Code Flow mit PKCE. Das Token geht nie über die Adressleiste des Browsers.

PKCE (*Proof Key for Code Exchange*) schützt davor, dass jemand den einmaligen Code aus Schritt 3 abfängt und selbst eintauscht: Der Client erzeugt vor Schritt 1 ein zufälliges Geheimnis (*code_verifier*), schickt nur dessen Hash mit (*code_challenge*) und beweist beim Eintausch in Schritt 4, dass er das Original kennt [11].

TIPP

Für Browser-Anwendungen beschreibt RFC 10017 (August 2026) das *Backend for Frontend* (BFF) als stärkstes der drei Muster [8]: Ein serverseitiger Teil führt den Flow aus und hält die Tokens, der Browser bekommt nur ein `Secure`- und `HttpOnly`-Session-Cookie. Das reduziert Token-Diebstahl durch JavaScript; gegen CSRF braucht das BFF zusätzlich einen eigenen Schutz (Kapitel 8) [8].

2.3 Maschine zu Maschine: Client Credentials

Ein Hintergrunddienst ohne Benutzer (etwa ein nächtlicher Export) meldet sich mit eigenen Zugangsdaten an und bekommt ein Token in eigenem Namen. Das ist der *Client Credentials Grant* [5]. In Authentik läuft dieser Weg über ein eigenes Dienstkonto mit App-Passwort oder über föderierte Tokens, die Details beschreibt die Authentik-Dokumentation unter *Machine-to-Machine (M2M) authentication* [12]. Wichtig für die API: Solche Tokens haben kein menschliches Subjekt und sollten nur genau die Rechte bekommen, die der Dienst braucht.

2.4 Authentik einrichten

In Authentik gehören zu jeder Anwendung zwei Objekte: ein **Provider** (Typ *OAuth2/OpenID Provider*) mit den technischen Einstellungen und eine **Application**, die den Provider mit einem Namen (*Slug*) und Zugriffsregeln verbindet. Für die Web-App der Notizen:

Einstellung	Empfehlung
Client type	<i>Confidential</i> für serverseitige Clients (BFF, Dienste), <i>Public</i> für reine Mobile- oder Desktop-Apps, die kein Geheimnis sicher speichern können [5], [10]
Redirect URIs	exakte, vollständige Adressen (<i>strict</i>), keine regulären Ausdrücke mit Platzhaltern [7]
Signing Key	ein Zertifikat auswählen (z.B. das mitgelieferte selbstsignierte). Ohne Auswahl signiert Authentik mit dem Client-Secret per HS256, dann kann die API die Tokens nicht über öffentliche Schlüssel prüfen [10].
Access token validity	kurz, etwa <code>minutes=5</code> . Kurze Laufzeiten begrenzen den Schaden gestohlener Tokens [7].
Refresh tokens	nur für Clients, die sie brauchen, mit begrenzter Laufzeit und Rotation [7]. Seit Authentik 2024.2 muss der Client <code>offline_access</code> anfordern und das Mapping im Provider aktiviert sein [10], [13].
Scopes	nur benötigte Scope-Mappings, etwa <code>openid</code> , <code>profile</code> , <code>email</code> und eigene wie <code>notizen:lesen</code>
Application -> Policy Bindings	festlegen, welche Gruppen sich überhaupt an dieser Anwendung anmelden dürfen

Die Tabelle gilt für den voreingestellten und empfohlenen **Issuer mode pro Provider** [10]. Beim globalen Issuer lautet `iss` nur `https://auth.example.com/`, während Discovery und JWKS weiterhin unter dem Application-Slug liegen [10]. Die Anwendung übernimmt `issuer` und `jwks_uri` deshalb aus dem Discovery-Dokument oder aus expliziter Konfiguration und setzt Adressen nicht durch String-Verkettung zusammen. Die für die API wichtigen Adressen leiten sich aus dem Slug der Application ab (hier `notizen`). Der *Issuer* endet mit einem Schrägstrich, der genau so im Token steht und genau so geprüft werden muss:

Zweck	Adresse [10]
Issuer (<code>iss</code>)	<code>https://auth.example.com/application/o/notizen/</code>
Discovery	<code>https://auth.example.com/application/o/notizen/.well-known/openid-configuration</code>
Öffentliche Schlüssel (JWKS)	<code>https://auth.example.com/application/o/notizen/jwks/</code>
Token-Endpunkt	<code>https://auth.example.com/application/o/token/</code>

2.5 Access-Token, ID-Token, Refresh-Token

Token	Zweck	Empfänger
Access-Token	berechtigt zum Zugriff auf die API	die API (Resource Server) [5]
ID-Token	sagt dem Client, wer sich angemeldet hat [6]	nur der Client, nie die API
Refresh-Token	holt neue Access-Tokens ohne erneute Anmeldung	nur der Authorization Server [5]

ACHTUNG

Eine API darf ausschließlich Access-Tokens akzeptieren. Ein ID-Token ist für den Client bestimmt und beweist nichts über Zugriffsrechte. Signatur, Issuer und Audience allein unterscheiden beide Token-Arten nicht immer sicher; Kapitel 3 erklärt den aktuellen Authentik-Grenzfall.

Authentik stellt Access-Tokens als signierte JSON Web Tokens (JWT) aus [10]. Ein JWT besteht aus drei Base64-kodierten Teilen, getrennt durch Punkte: Kopf (Algorithmus und Schlüssel-ID `kid`), Nutzlast (Claims) und Signatur [14]. Die Nutzlast ist **nur kodiert, nicht verschlüsselt**: Jeder, der das Token hat, kann sie lesen. Deshalb gehören keine Geheimnisse in Tokens [14].

Nutzlast eines Access-Tokens (gekürzt)

```
{
  "iss": "https://auth.example.com/application/o/notizen/",
  "sub": "3f7c2e1a9b...",
  "aud": "ZkP8x4Qm...",
  "exp": 1790253000,
  "iat": 1790252700,
  "scope": "openid profile notizen:lesen notizen:schreiben",
  "groups": ["notizen-admins"]
}
```

Welche Claims genau enthalten sind, bestimmen die Scope-Mappings des Providers. Gruppen landen über das Standard-Mapping des Scopes `profile` im Claim `groups` [10]. `aud` (*audience*) ist bei Authentik typischerweise die Client-ID des Providers [15]. Für eine belastbare Trennung erhält die API einen eigenen Provider beziehungsweise eine eigene Audience; Authentik 2026.8 kann über *Token Exchange* (RFC 8693) ein Token für diesen Ziel-Provider ausstellen [16], [17]. Token Exchange muss ausdrücklich als Grant Type aktiviert und auf vertrauenswürdige Provider begrenzt werden; die On-behalf-of-Delegation gibt es seit 2026.8, und sie ist mit diesem Grant Type automatisch verfügbar [17].

3 Tokens in FastAPI prüfen

Token-Prüfung ist mehr als Dekodieren: Die API muss kryptografisch und semantisch beweisen, dass das Token vom erwarteten Aussteller stammt, für genau diese API bestimmt, noch gültig und tatsächlich ein Access-Token ist [18].

3.1 Der Prüfvertrag

Prüfung	Warum
Signatur mit einem bekannten öffentlichen Schlüssel	Nur der erwartete Authorization Server kann gültige Tokens erzeugen [18].
Algorithmus aus einer festen Liste	Verhindert <code>alg: none</code> und Algorithmus-Verwechslungen [18].
Issuer (<code>iss</code>) exakt	Kein Token eines anderen Providers derselben Instanz [18].
Audience (<code>aud</code>) der API	Kein Token für eine andere Anwendung [18].
Typ des Tokens	ID-Token und andere JWT-Arten dürfen nie als Access-Token gelten [18], [19].
Zeit (<code>exp</code> , ggf. <code>nbf</code> , <code>iat</code>)	Abgelaufene oder noch nicht gültige Tokens werden abgewiesen; kleine Toleranz nur für Uhrabweichung [14].
Claim-Formate	<code>sub</code> , <code>scope</code> und <code>groups</code> haben den erwarteten Typ und Inhalt.
Berechtigungen	Scopes und Objekt-/Funktionsrechte werden pro Endpunkt geprüft.

RFC 8725 verlangt feste Algorithmen und voneinander getrennte Validierungsregeln für unterschiedliche JWT-Arten [18]. RFC 9068 definiert für standardkonforme JWT-Access-Tokens den Header `typ: at+jwt` und eine API-spezifische Audience [19].

ACHTUNG

Stand Authentik 2026.8: Authentik erzeugt noch keine RFC-9068-konformen Access-Tokens mit `typ: at+jwt` [15]. Ein lokal geprüftes Authentik-JWT lässt sich daher nicht allein anhand des Standard-Headers zweifelsfrei von einem ID-Token unterscheiden. Das ist keine Kleinigkeit: Ist die Audience gleich, kann ein gültiges ID-Token sonst den lokalen Prüfvertrag erfüllen [15].

Für Authentik gibt es drei belastbare Betriebsvarianten:

1. **API-spezifischer Provider/Audience** und Token Exchange: Die API akzeptiert nur Tokens des Ziel-Providers [17]; ein Integrationstest muss ein echtes ID-Token ausdrücklich ablehnen.
2. **Opaque Token beziehungsweise Introspection:** Die API fragt den vertraulichen Provider am Introspection-Endpunkt nach Aktivität und Token-Metadaten [10], [20]. Das erleichtert Widerruf, kostet aber Verfügbarkeit und Latenz.
3. **Lokaler, installationsspezifischer Access-Token-Claim:** etwa `token_use=access`, der nachweislich nur im Access-Token vorkommt. Das ist kein Standardsatz für RFC 9068 und muss nach jedem IdP-Update mit echten Token-Proben getestet werden.

3.2 Lokale JWT-Prüfung

`jwtks_url` stammt aus `jwtks_uri` des Discovery-Dokuments und wird als Konfiguration übernommen. So funktioniert auch der globale Authentik-Issuer-Modus, bei dem Issuer und JWKS-Adresse nicht denselben Pfad haben.

app/auth.py

```
import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from jwt import PyJWKClient
from jwt.exceptions import PyJWKClientError, PyJWTError
from pydantic import BaseModel, ConfigDict, Field, ValidationError, field_validator

from app.settings import settings

_jwtks = PyJWKClient(
    settings.jwtks_url,
    cache_jwk_set=True,
    lifespan=300,
    timeout=5,
    cooldown_duration=30,
)
_bearer = HTTPBearer(auto_error=False)

class TokenClaims(BaseModel):
    model_config = ConfigDict(extra="ignore")
    sub: str
    scope: str = ""
    groups: list[str] = Field(default_factory=list)
    token_use: str

    @field_validator("sub")
    @classmethod
    def sub_nicht_leer(cls, wert: str) -> str:
        if not wert:
            raise ValueError("sub fehlt")
        return wert

class Benutzer(BaseModel):
    id: str
    scopes: frozenset[str]
    gruppen: frozenset[str]

def _nicht_angemeldet() -> HTTPException:
    return HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Nicht angemeldet",
        headers={"WWW-Authenticate": "Bearer"},
    )

def aktueller_benutzer(
    cred: HTTPAuthorizationCredentials | None = Depends(_bearer),
) -> Benutzer:
    if cred is None or cred.scheme.lower() != "bearer":
        raise _nicht_angemeldet()
    try:
        schluessel = _jwtks.get_signing_key_from_jwt(cred.credentials)
        roh = jwt.decode(
            cred.credentials,
            schluessel.key,
            algorithms=["RS256"],
            issuer=settings.issuer,
            audience=settings.audience,
```

```

        leeway=30,
        options={"require": ["exp", "iat", "iss", "aud", "sub"]},
    )
    claims = TokenClaims.model_validate(roh)
    if claims.token_use != "access": # eigenes Authentik-Scope-Mapping
        raise ValueError("kein Access-Token")
    except (PyJWTError, PyJWKClientError, ValidationError, ValueError):
        raise _nicht_angemeldet() from None
    return Benutzer(
        id=claims.sub,
        scopes=frozenset(claims.scope.split()),
        gruppen=frozenset(claims.groups),
    )

```

Der Claim `token_use` ist hier ein **bewusstes lokales Vertragsmerkmal**. Das zugehörige Authentik-Scope-Mapping darf nicht ins ID-Token übernommen werden. Ein Test mit real ausgestellten Access- und ID-Tokens ist Teil des Deployments. Sobald Authentik RFC 9068 unterstützt, wird stattdessen vor dem Dekodieren der unverifizierte Header gelesen, ausschließlich `typ` geprüft und nur `at+jwt` beziehungsweise `application/at+jwt` akzeptiert [19].

`PyJWKClient` hält in dieser Konfiguration das **JWKS-Set** fünf Minuten im Cache [21]. `cache_keys=True` wird bewusst nicht verwendet: Diese Option würde einzelne Schlüssel zusätzlich ohne Zeitablauf per LRU cachen. Eine unbekannte `kid` löst, durch eine Abkühlzeit gegen Missbrauch begrenzt, eine Aktualisierung aus [22]. Ist der IdP nicht erreichbar und kein passender Schlüssel verfügbar, gilt *fail closed*: 401.

Alle Systeme synchronisieren ihre Uhr. `leeway=30` ist eine kleine, begründete Toleranz, keine Verlängerung der Token-Laufzeit [14]. Die API folgt niemals `jku` oder `x5u` aus einem Token [18]; der JWKS-Endpunkt kommt ausschließlich aus vertrauenswürdiger Konfiguration.

3.3 Scopes und Gruppen prüfen

app/auth.py (Fortsetzung)

```

def braucht_scope(*erforderlich: str):
    def pruefen(benutzer: Benutzer = Depends(aktueller_benutzer)) -> Benutzer:
        if not set(erforderlich) <= benutzer.scopes:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Keine Berechtigung")
        return benutzer
    return pruefen

def braucht_gruppe(gruppe: str):
    def pruefen(benutzer: Benutzer = Depends(aktueller_benutzer)) -> Benutzer:
        if gruppe not in benutzer.gruppen:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Keine Berechtigung")
        return benutzer
    return pruefen

```

app/main.py (Ausschnitt)

```

from fastapi import APIRouter, Depends, FastAPI
from app.auth import Benutzer, aktueller_benutzer, braucht_scope

app = FastAPI()
notizen = APIRouter(prefix="/notizen", dependencies=[Depends(aktueller_benutzer)])

@notizen.get("")
def liste(benutzer: Benutzer = Depends(braucht_scope("notizen:lesen"))): ...

@notizen.post("")
def anlegen(benutzer: Benutzer = Depends(braucht_scope("notizen:schreiben"))): ...

app.include_router(notizen)

```

Die Router-Abhängigkeit schützt neue Endpunkte standardmäßig; öffentliche Endpunkte gehören auf einen eigenen Router. FastAPI cached denselben Dependency-Aufruf innerhalb einer Anfrage, daher wird `aktueller_benutzer` trotz der zusätzlichen Scope-Abhängigkeit nicht doppelt ausgeführt [23].

3.4 Widerruf und gebundene Tokens

Ein lokal geprüftes JWT bleibt grundsätzlich bis `exp` gültig [14]. Deshalb sind Access-Tokens kurzlebig. Für sofortigen Widerruf wird Introspection mit kurzem, begrenztem Cache eingesetzt [20]; Client-Credentials dafür liegen als Secret vor, Fehler schließen den Zugriff. Öffentliche Clients müssen Refresh-Tokens nach RFC 9700 rotieren oder sendergebunden einsetzen [7]. Authentik rotiert Refresh-Tokens automatisch, wenn es im Provider konfiguriert ist [10].

Für Hochrisiko-APIs können mTLS-gebundene Tokens (RFC 8705) oder DPoP (RFC 9449) den Token-Diebstahl erschweren [24], [25]. Das Verfahren muss Access-Token, Client und API durchgängig unterstützen; ein `bound_key`-Scope, der nur ein ID-Token bindet, genügt dafür nicht [10].

4 Autorisierung: Objekte und Funktionen

Authentifizierung beantwortet „Wer bist du?“, Autorisierung „Darfst du **das**?“. Die OWASP-Liste führt zwei Autorisierungsfehler auf den Plätzen 1 und 5, weil sie so häufig und so folgenreich sind: Zugriff auf fremde **Objekte** (BOLA) [1] und Zugriff auf fremde **Funktionen** [26].

4.1 BOLA: Broken Object Level Authorization

Das Muster ist fast immer dasselbe: Ein Endpunkt bekommt die ID eines Objekts, prüft, dass jemand angemeldet ist, und liefert das Objekt aus, ohne zu prüfen, ob es diesem Jemand gehört [1].

app/main.py

```
# UNSICHER: jeder angemeldete Benutzer kann jede Notiz lesen
@notizen.get("/{notiz_id}")
def lesen(notiz_id: UUID, benutzer: Benutzer = Depends(braucht_scope("notizen:lesen"))):
    notiz = db.execute(select(Notiz).where(Notiz.id == notiz_id)).scalar_one_or_none()
    if notiz is None:
        raise HTTPException(404)
    return notiz
```



Abb. 3: BOLA: Die Anmeldung ist korrekt, die Frage nach dem Besitz fehlt.

Die richtige Lösung ist, die Besitzprüfung **in die Abfrage selbst** zu legen. Dann kann es gar nicht passieren, dass ein fremdes Objekt überhaupt geladen wird:

app/main.py

```
@notizen.get("/{notiz_id}", response_model=NotizAusgabe)
def lesen(notiz_id: UUID, benutzer: Benutzer = Depends(braucht_scope("notizen:lesen"))):
    notiz = db.execute(
        select(Notiz).where(Notiz.id == notiz_id, Notiz.besitzer_id == benutzer.id)
    ).scalar_one_or_none()
    if notiz is None:
        raise HTTPException(404, "Nicht gefunden") # auch bei fremden Notizen: 404
    return notiz
```

Drei Punkte dazu:

- **404 statt 403** für fremde Objekte. Ein 403 verrät, dass es das Objekt gibt; HTTP erlaubt es ausdrücklich, die Existenz eines Objekts mit 404 zu verbergen [27]. Das hilft beim Ausspähen, etwa von E-Mail-Adressen oder Kundennummern.
- **Zufällige IDs (UUIDs) sind kein Schutz.** Sie erschweren das Raten, aber IDs gelangen über geteilte Links, Logs, Referer-Header oder andere Endpunkte nach außen. Die Prüfung muss trotzdem stattfinden [1], [28].
- **Jeder Zugriffsweg braucht die Prüfung:** Lesen, Ändern, Löschen, Exportieren, Anhänge herunterladen, Unterobjekte (/notizen/{id}/kommentare) [1]. Besonders leicht vergessen werden Massenoperationen und Suchendpunkte.

Autorisierung zentralisieren

Wenn Notizen geteilt werden können, wird die Regel komplexer: lesen darf der Besitzer, wer eine Freigabe hat, und ein Administrator. Solche Regeln gehören an **eine** Stelle, nicht verstreut in jeden Endpunkt [29]:

app/rechte.py

```

from sqlalchemy import or_, select

def sichtbare_notizen(benutzer: Benutzer):
    """Basisabfrage: alle Notizen, die dieser Benutzer lesen darf."""
    abfrage = select(Notiz)
    if "notizen-admins" in benutzer.gruppen:
        return abfrage
    freigegeben = select(Freigabe.notiz_id).where(Freigabe.benutzer_id == benutzer.id)
    return abfrage.where(or_(Notiz.besitzer_id == benutzer.id, Notiz.id.in_(freigegeben)))

def darf_aendern(benutzer: Benutzer, notiz: Notiz) -> bool:
    return notiz.besitzer_id == benutzer.id

```

Jeder Endpunkt beginnt dann mit `sichtbare_notizen(benutzer)` und schränkt weiter ein. Eine Änderung der Regel wirkt überall gleichzeitig.

TIPP

Als zusätzliche Verteidigungslinie kann PostgreSQL die Besitzregel selbst durchsetzen: Mit *Row Level Security* (`ALTER TABLE notizen ENABLE ROW LEVEL SECURITY` und einer Policy auf eine Sitzungsvariable mit der Benutzer-ID) liefert die Datenbank fremde Zeilen gar nicht erst aus [30], auch wenn im Code eine Prüfung fehlt. Der Aufwand lohnt sich bei mandantenfähigen Anwendungen mit vielen Endpunkten.

4.2 Autorisierung auf Funktionsebene

Der zweite Klassiker: Administrative Funktionen sind nur in der Oberfläche versteckt, die API prüft aber nicht, wer sie aufruft. Wer die Endpunkte kennt (etwa aus der öffentlichen OpenAPI-Dokumentation oder dem JavaScript der Web-App), ruft sie einfach direkt auf [26].

app/admin.py

```

admin = APIRouter(
    prefix="/admin",
    dependencies=[Depends(braucht_gruppe("notizen-admins"))], # für ALLE Endpunkte darunter
)

@admin.delete("/benutzer/{benutzer_id}/notizen")
def alle_loeschen(benutzer_id: str): ...

```

Die Regel ist dieselbe wie bei der Anmeldung: Rechte auf Router-Ebene vergeben, damit neue Endpunkte automatisch geschützt sind [31]. Achtung auch bei HTTP-Methoden [26]: Ist `GET /notizen/{id}` korrekt geschützt, heißt das nichts über `PUT`, `PATCH` und `DELETE` auf derselben Adresse.

4.3 Autorisierung testen

Autorisierungsfehler findet kein Scanner, aber ein einfacher Test mit zwei Benutzern findet fast alle. FastAPI erlaubt, die Anmelde-Abhängigkeit im Test durch feste Benutzer zu ersetzen [32]:

tests/test_rechte.py

```

from contextlib import contextmanager
import pytest
from fastapi.testclient import TestClient
from app.main import app
from app.auth import Benutzer, aktueller_benutzer

ALICE = Benutzer(id="alice", scopes=frozenset({"notizen:lesen", "notizen:schreiben"}),
gruppen=frozenset())
BOB = Benutzer(id="bob", scopes=frozenset({"notizen:lesen", "notizen:schreiben"}),
gruppen=frozenset())

@pytest.fixture

```

```

def client():
    with TestClient(app) as testclient:
        yield testclient

@contextmanager
def als(benutzer: Benutzer):
    vorher = app.dependency_overrides.get(aktueller_benutzer)
    app.dependency_overrides[aktueller_benutzer] = lambda: benutzer
    try:
        yield
    finally:
        if vorher is None:
            app.dependency_overrides.pop(aktueller_benutzer, None)
        else:
            app.dependency_overrides[aktueller_benutzer] = vorher

def test_bob_sieht_notiz_von_alice_nicht(client):
    with als(ALICE):
        notiz_id = client.post("/notizen", json={"titel": "privat", "text": "..."}).json()["id"]
    with als(BOB):
        antwort = client.get(f"/notizen/{notiz_id}")
    assert antwort.status_code == 404

@pytest.mark.parametrize("methode", ["put", "patch", "delete"])
def test_bob_kann_notiz_von_alice_nicht_aendern(client, methode):
    with als(ALICE):
        notiz_id = client.post("/notizen", json={"titel": "x", "text": "y"}).json()["id"]
    with als(BOB):
        antwort = client.request(methode.upper(), f"/notizen/{notiz_id}",
                                json={"titel": "gehackt"})
    assert antwort.status_code == 404

```

Der Override wird nach jedem Block sicher zurückgesetzt; sonst beeinflusst ein Test den nächsten. Ein 405 wäre hier kein Erfolg: Er würde nur beweisen, dass die Methode fehlt, nicht dass ihre Objektberechtigung stimmt. Für jeden real vorhandenen Lese-, Änderungs-, Lösch-, Export- und Unterobjekt-Endpunkt gehört ein exakter „fremder Benutzer“-Test zur Definition of Done.

MERKE

In mandantenfähigen Systemen gehört neben der Besitzer-ID immer auch die Mandanten-ID in jede Abfrage. Bei PostgreSQL-RLS werden Mandant und Benutzer transaktionslokal gesetzt [33]; der Anwendungs-DB-Account darf die Policy weder umgehen noch Tabellen besitzen [30]. RLS ergänzt die Anwendungstests, ersetzt sie aber nicht.

5 Datenmodelle: was hinein und was hinaus darf

Platz 3 der OWASP-Liste, *Broken Object Property Level Authorization*, fasst zwei früher getrennte Risiken zusammen: Eine API liefert **mehr Felder** aus, als der Aufrufer sehen darf (*Excessive Data Exposure*), oder sie übernimmt **mehr Felder** aus der Anfrage, als der Aufrufer setzen darf (*Mass Assignment*) [34]. Beides lässt sich mit getrennten, expliziten Pydantic-Modellen fast vollständig ausschließen.

5.1 Ein Modell pro Richtung

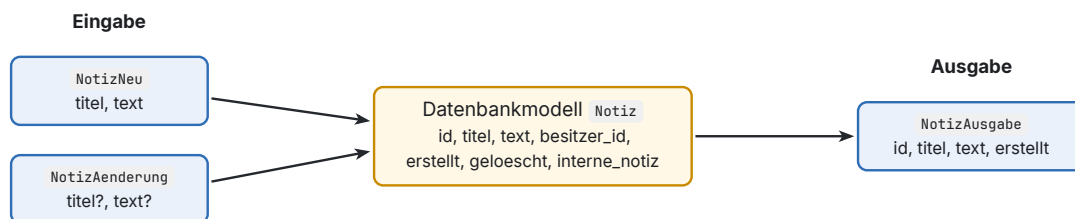


Abb. 4: Eingabe- und Ausgabemodelle enthalten nur erlaubte Felder, das Datenbankmodell bleibt intern.

app/modelle.py

```

from datetime import datetime
from uuid import UUID
from pydantic import BaseModel, ConfigDict, Field

class Eingabe(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

class NotizNeu(Eingabe):
    titel: str = Field(min_length=1, max_length=200)
    text: str = Field(max_length=20_000)

class NotizAenderung(Eingabe):
    titel: str | None = Field(default=None, min_length=1, max_length=200)
    text: str | None = Field(default=None, max_length=20_000)

class NotizAusgabe(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: UUID
    titel: str
    text: str
    erstellt: datetime
  
```

- `extra="forbid"` lässt jede Anfrage mit unbekannten Feldern scheitern (422) [35], [36]. Ein Angreifer, der `"besitzer_id": "..."` oder `"ist_admin": true` mitschickt, bekommt einen Fehler, statt dass das Feld stillschweigend ignoriert oder, schlimmer, übernommen wird.
- **Kein `**daten.model_dump()` direkt ins Datenbankmodell.** Felder wie `besitzer_id` setzt der Server aus dem Token, nie aus der Anfrage [34].
- **Das Ausgabemodell** legt über `response_model=NotizAusgabe` fest, welche Felder den Server verlassen [37]. Kommt später ein internes Feld zum Datenbankmodell hinzu, taucht es nicht automatisch in der API auf.

app/main.py

```
@notizen.patch("/{notiz_id}", response_model=NotizAusgabe)
def aendern(notiz_id: UUID, daten: NotizAenderung,
            benutzer: Benutzer = Depends(braucht_scope("notizen:schreiben"))):
    notiz = eigene_notiz_oder_404(notiz_id, benutzer)
    for feld, wert in daten.model_dump(exclude_unset=True).items():
        setattr(notiz, feld, wert)      # nur Felder, die NotizAenderung überhaupt kennt
    return notiz
```

Unterscheiden sich die Rechte **pro Feld** (Administratoren dürfen `interne_notiz` sehen, normale Benutzer nicht), bekommen die Rollen eigene Ausgabemodelle, statt Felder im Code dynamisch auszublenden.

5.2 Eingaben begrenzen und validieren

Jedes Feld braucht eine Obergrenze: Länge von Zeichenketten, Anzahl von Listenelementen, Wertebereich von Zahlen [38]. Ohne sie kann ein einzelner Request mit einem 500-MB-Textfeld oder einer Liste mit einer Million Einträgen Speicher und Datenbank überlasten (Kapitel 6).

Pydantic-Mittel	Einsatz
<code>Field(max_length=...), min_length</code>	Zeichenketten und Listen [39]
<code>Field(ge=1, le=100)</code>	Zahlen, z.B. Seitengröße [39]
<code>Literal["privat", "team"], Enum</code>	feste Wertelisten statt freier Zeichenketten
<code>EmailStr, HttpUrl, UUID</code>	Formate, die Pydantic selbst prüft [40]
<code>Field(pattern=r"^[a-z0-9-]{3,40}\$")</code>	Kennungen mit festem Aufbau [39]
<code>model_config = ConfigDict(strict=True)</code>	keine automatische Umwandlung, etwa von <code>"1"</code> in <code>1</code> [41]

Die Validierung ersetzt keine sichere Weiterverarbeitung: Datenbankabfragen laufen **immer** parametrisiert [42] (SQLAlchemy macht das automatisch, solange keine Zeichenketten zu SQL zusammengesetzt werden [43]), Ausgaben in HTML werden escaped [44], Dateinamen aus Anfragen nie direkt als Pfad verwendet [45].

ACHTUNG

`text(f"SELECT * FROM notizen WHERE titel = '{titel}'")` ist auch mit SQLAlchemy eine SQL-Injection [46]. Richtig: `text("SELECT * FROM notizen WHERE titel = :titel").bindparams(titel=titel)` oder die Abfrage-API von SQLAlchemy.

5.3 Datei-Uploads

- Größe begrenzen, bevor die Datei vollständig eingelesen wird (am Reverse Proxy und in der Anwendung) [45].
- Dateityp am Inhalt prüfen, nicht an Endung oder `Content-Type` des Clients; nur eine kleine Allowlist zulassen [45].
- Erst in **Quarantäne** unter einem zufälligen, serverseitig erzeugten Namen speichern. Vor Freigabe Malware-Scan und – falls Archive erlaubt sind – Grenzen für Dateianzahl, entpackte Größe und Verschachtelung anwenden [45].
- Außerhalb des Webroots oder in privatem Object Storage speichern [45]. Downloads autorisieren, kurzlebige signierte URLs verwenden [47] und möglichst über eine eigene Download-Domain ohne Cookies ausliefern.
- Beim Ausliefern `Content-Disposition: attachment` mit sicher kodiertem Dateinamen [48], `X-Content-Type-Options: nosniff` [49] und einen serverseitig bestimmten Typ setzen. Benutzerdateinamen werden nie als Pfad oder Header-Rohwert übernommen.

Nicht jedes Format ist gleich: SVG, HTML, Office-Dokumente und PDFs können aktive Inhalte oder externe Referenzen enthalten [45], [50]. Wenn die Funktion sie nicht zwingend braucht, werden sie nicht akzeptiert; sonst erfolgt eine formatspezifische Bereinigung beziehungsweise Konvertierung in einer isolierten Umgebung.

6 Ressourcen und Geschäftsabläufe schützen

Eine API ohne Grenzen lädt zu zwei Arten von Missbrauch ein: Sie lässt sich mit Anfragen überlasten (OWASP API4, *Unrestricted Resource Consumption*) [38], und legitime Abläufe lassen sich automatisiert ausnutzen, etwa massenhaftes Registrieren, Durchprobieren von Gutscheincodes oder das Leerkufen limitierter Angebote (API6, *Unrestricted Access to Sensitive Business Flows*) [51].

6.1 Grenzen auf allen Ebenen

Grenze	Umsetzung
Größe einer Anfrage	am Reverse Proxy (Caddy: <code>request_body { max_size 1MB }</code>) und in den Pydantic-Modellen [38], [52]
Anzahl der Ergebnisse	Pflicht-Paginierung mit Höchstwert: <code>limit: int = Query(20, ge=1, le=100)</code> [38], [53]
Anfragen pro Zeit	Rate Limiting pro Benutzer bzw. Client, zusätzlich pro IP für nicht angemeldete Endpunkte [38]
Laufzeit	Timeouts für Datenbankabfragen (<code>statement_timeout</code> in PostgreSQL) und ausgehende HTTP-Aufrufe [54]
Teure Operationen	Exporte, Berichte, Suchen mit Platzhaltern: eigene, strengere Limits oder asynchron als Job [38]
Speicher und CPU	Container-Limits (Docker-Handbuch, Kapitel 12) [38]

6.2 Rate Limiting

Rate Limiting begrenzt, wie viele Anfragen ein Aufrufer in einem Zeitfenster stellen darf. Überschreitet er das Limit, antwortet die API mit `429 Too Many Requests` [55] und kann im Header `Retry-After` angeben, wann ein neuer Versuch sinnvoll ist [27]. Für FastAPI eignet sich die Bibliothek `slowapi`, die ihre Zähler bei mehreren API-Instanzen in Redis ablegt [56]. Damit sie `Retry-After` mitschickt, wird der Limiter mit `headers_enabled=True` angelegt [57]:

app/limits.py

```
from fastapi import Request
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded

def schluessel(request: Request) -> str:
    # angemeldete Anfragen pro Benutzer zählen, sonst pro IP
    benutzer = getattr(request.state, "benutzer_id", None)
    return f"u:{benutzer}" if benutzer else f"ip:{request.client.host}"

limiter = Limiter(key_func=schluessel, storage_uri="redis://redis:6379/0",
                  default_limits=["300/minute"], headers_enabled=True)
```

app/main.py

```
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@notizen.post("/export")
@limiter.limit("5/hour")                                # teure Operation: eigenes, strenges Limit
```

```
def export(request: Request, benutzer: Benutzer = Depends(braucht_scope("notizen:lesen"))):
    ...
```

ACHTUNG

Hinter einem Reverse Proxy sieht die Anwendung als `request.client.host` die Adresse des Proxys, alle Clients teilen sich dann ein Limit [58]. Der Server muss den Headern des Proxys (`X-Forwarded-For`) vertrauen, aber **nur** denen des Proxys: `fastapi run` bzw. `Uvicorn` übernehmen sie nur von Adressen, die in `FORWARDED_ALLOW_IPS` stehen (Standard: nur `127.0.0.1` und `::1`) [59], [60]. Dort gehört die Adresse bzw. das Netz des Proxys hinein, niemals * bei direkt erreichbaren Servern, sonst kann jeder Client seine IP frei wählen [58].

Damit der Schlüssel pro Benutzer funktioniert, erweitert man `aktueller_benutzer` aus Kapitel 3 um den Parameter `request: Request` und setzt dort `request.state.benutzer_id = claims["sub"]`.

Ein zweites, grobes Limit am Reverse Proxy fängt Lastspitzen ab, bevor sie Python erreichen. Fehlversuche bei der Anmeldung bewertet Authentik selbst: Eine Reputation-Policy zählt sie pro IP-Adresse oder Benutzername und kann bei geringem Vertrauen zusätzliche Prüfungen wie ein CAPTCHA verlangen, sie muss dazu aber in den Anmelde-Flows eingerichtet werden [61].

6.3 Idempotenz für kritische Abläufe

Netzwerke sind unzuverlässig: Ein Client schickt eine Bestellung, die Antwort geht verloren, der Client wiederholt die Anfrage. Ohne Vorkehrung entsteht die Bestellung zweimal. Für solche Abläufe schickt der Client einen eindeutigen Schlüssel im Header `Idempotency-Key` mit (ein Entwurf der IETF, kein verabschiedeter Standard) [62]. Die API merkt sich Schlüssel und Antwort und liefert bei einer Wiederholung die gespeicherte Antwort, statt erneut auszuführen:

`app/idempotenz.py` (Skizze)

```
from sqlalchemy.dialects.postgresql import insert

@app.post("/bestellungen", status_code=201)
def bestellen(daten: BestellungNeu, idempotency_key: UUID = Header(),
             benutzer: Benutzer = Depends(braucht_scope("bestellen"))):
    fingerabdruck = hashlib.sha256(daten.model_dump_json().encode()).hexdigest()
    with db.begin():
        neu = db.execute(
            insert(Idempotenz).values(
                benutzer_id=benutzer.id, schluessel=idempotency_key,
                fingerabdruck=fingerabdruck, status="laeuft",
            ).on_conflict_do_nothing(
                index_elements=["benutzer_id", "schluessel"]
            ).returning(Idempotenz.id)
        ).scalar_one_or_none()
    if neu is None:
        eintrag = idempotenz_lesen(benutzer.id, idempotency_key, for_update=True)
        if eintrag.fingerabdruck != fingerabdruck:
            raise HTTPException(422, "Schlüssel gehört zu einer anderen Anfrage")
        if eintrag.status == "fertig":
            return gespeicherte_antwort(eintrag) # Statuscode, Header, Body
        raise HTTPException(409, "Anfrage wird bereits verarbeitet",
                           headers={"Retry-After": "2"})
    antwort = bestellung_anlegen(daten, benutzer)
    idempotenz_abschliessen(neu, antwort)
    return antwort
```

Die Datenbank erzwingt einen Unique-Constraint auf `(benutzer_id, schluessel)`. Die atomare Reservierung verhindert, dass zwei gleichzeitige Requests beide die Wirkung ausführen [63], [64] – ein vorheriges `SELECT` mit anschließendem `INSERT` würde das nicht leisten. Fingerabdruck, Geschäftswirkung und gespeicherte Antwort (Statuscode, relevante Header und Body) liegen in derselben Transaktion; externe Wirkungen werden über eine transaktionale Outbox angestoßen [65]. Abgelaufene Einträge werden erst

nach einer dokumentierten Wiederholungsfrist gelöscht. Nach dem Entwurf antwortet der Server auf einen Schlüssel, der mit anderer Nutzlast wiederverwendet wird, mit 422 und auf eine noch laufende Anfrage mit 409 [62].

6.4 Missbrauch legitimer Abläufe

Gegen automatisierten Missbrauch von Geschäftsabläufen hilft kein einzelnes technisches Mittel. Zuerst gilt es, die gefährdeten Abläufe zu identifizieren [51]: Was passiert, wenn jemand diesen Endpunkt 10.000-mal pro Stunde aufruft? Typische Gegenmaßnahmen sind strengere Limits pro Konto und Zeitraum (etwa höchstens drei Freigabe-Einladungen pro Minute), Verzögerungen nach Fehlversuchen, zusätzliche Bestätigung per E-Mail oder zweitem Faktor bei kritischen Aktionen und das Überwachen auffälliger Muster in den Logs (Kapitel 7).

7 Fehler, Logging und Datensparsamkeit

Fehlermeldungen und Logs sind für Entwickler gemacht, werden aber leicht zu einer Informationsquelle für Angreifer [66] oder zu einem Datenschutzproblem. Das Ziel: Der Client erfährt genug, um sinnvoll zu reagieren, aber nichts über Interna. Die Logs enthalten genug, um Vorfälle aufzuklären, aber keine Geheimnisse und so wenig personenbezogene Daten wie möglich.

7.1 Einheitliche Fehler nach RFC 9457

RFC 9457 (*Problem Details for HTTP APIs*, Nachfolger von RFC 7807) definiert ein einheitliches JSON-Format für Fehler mit dem Medientyp `application/problem+json` [67]:

```
{
  "type": "https://api.example.com/problems/nicht-gefunden",
  "title": "Nicht gefunden",
  "status": 404,
  "detail": "Die angeforderte Notiz existiert nicht.",
  "instance": "/notizen/7f3e0c1a-...",
  "request_id": "b3a1f2e4"
}
```

app/fehler.py

```
import logging, uuid
from http import HTTPStatus
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

log = logging.getLogger("notizen")

def problem(status: int, titel: str, request: Request, detail: str | None = None,
            headers: dict[str, str] | None = None):
    return JSONResponse(
        status_code=status,
        media_type="application/problem+json",
        content={"title": titel, "status": status, "detail": detail,
                 "instance": request.url.path, "request_id": request.state.request_id},
        headers=headers,
    )

def registrieren(app: FastAPI) -> None:
    @app.exception_handler(StarletteHTTPException)
    async def http_fehler(request: Request, exc: StarletteHTTPException):
        titel = next((s.phrase for s in HTTPStatus if s.value == exc.status_code),
                     "HTTP-Fehler")
        detail = exc.detail if isinstance(exc.detail, str) else None
        return problem(exc.status_code, titel, request, detail, exc.headers)

    @app.exception_handler(RequestValidationError)
    async def validierung(request: Request, exc: RequestValidationError):
        felder = [".".join(map(str, e["loc"])) for e in exc.errors()]
        return problem(422, "Ungültige Eingabe", request, f"Betroffene Felder: {'',
        '.join(felder)}")
```

```
@app.exception_handler(Exception)
async def unerwartet(request: Request, exc: Exception):
    log.exception("Unerwarteter Fehler", extra={"request_id": request.state.request_id})
    return problem(500, "Interner Fehler", request) # kein Stacktrace an den Client
```

Der Validierungs-Handler gibt nur Feldnamen zurück, nicht die eingegebenen Werte [36]. Der HTTP-Handler übernimmt `exc.headers`; sonst ginge etwa das für 401 erforderliche `WWW-Authenticate` [27] verloren. Der generische Titel kommt aus dem Statuscode, während Details nur aus bewusst erzeugten Anwendungsfehlern stammen. Der letzte Handler fängt Unerwartetes: Der Client bekommt nur die `request_id` [67], [68], mit der sich der vollständige Fehler im Log finden lässt.

7.2 Eine Request-ID für jede Anfrage

app/main.py

```
@app.middleware("http")
async def request_id(request: Request, call_next):
    request.state.request_id = uuid.uuid4().hex[:12]
    antwort = await call_next(request)
    antwort.headers["X-Request-ID"] = request.state.request_id
    return antwort
```

Die ID erscheint in jeder Logzeile und in jeder Fehlerantwort. Meldet ein Benutzer einen Fehler, genügt die ID, um den Ablauf nachzuvollziehen [69], ohne dass er Details beschreiben muss.

7.3 Was ins Log gehört und was nicht

Loggen	Nie loggen
Zeitpunkt, Request-ID, Methode, Pfad (ohne Query-Werte mit Geheimnissen), Statuscode, Dauer	Access-Tokens, Refresh-Tokens, Passwörter, API-Schlüssel, Client-Secrets [69]
Benutzer-ID (<code>sub</code>), nicht Name oder E-Mail	vollständige Request- und Response-Bodies
Sicherheitsereignisse: 401, 403, 429, abgelehnte Tokens, Rechteänderungen [69]	Gesundheits-, Finanz- oder andere besonders schützenswerte Inhalte [69]
Ursache unerwarteter Fehler mit Stacktrace (nur serverseitig)	den <code>Authorization</code> -Header, auch nicht „zur Fehlersuche“

Logs werden strukturiert (JSON) mit UTC-Zeit geschrieben, damit sie sich durchsuchen und auswerten lassen. Ein eigenes *Audit-Log* für sicherheitsrelevante Aktionen (Freigabe erteilt, Notiz gelöscht, Rolle geändert) mit Akteur, Aktion, Objekt, Ergebnis und Zeitpunkt ist bei Vorfällen oft die einzige Möglichkeit festzustellen, was geschehen ist. Es ist zugriffsbeschränkt, gegen nachträgliche Änderung geschützt und hat eine festgelegte Aufbewahrungs- und Löschfrist [69]. Für verteilte Systeme ergänzt W3C Trace Context [70] beziehungsweise OpenTelemetry [71] die lokale Request-ID; externe IDs werden validiert [70] und nie blind als Logstruktur übernommen.

MERKE

Datensparsamkeit [72] ist die wirksamste Datenschutzmaßnahme: Was nicht gespeichert wird, kann weder abfließen noch muss es gelöscht, beauskunftet oder geschützt werden. Für jede Spalte, jedes Log-Feld und jedes Feld in einer Antwort lohnt die Frage, ob es wirklich gebraucht wird. Pseudonyme IDs aus dem IdP (`sub`) statt E-Mail-Adressen in der eigenen Datenbank sind ein einfacher erster Schritt.

8 Transport, Header, CORS und Konfiguration

OWASP API8, *Security Misconfiguration*, ist ein Sammelbecken für Fehler, die nicht im Code, sondern in der Konfiguration stecken: unverschlüsselte Verbindungen, fehlende Header, zu offene CORS-Regeln, aktivierter Debug-Modus, öffentlich erreichbare Dokumentation [66]. Einzeln harmlos wirkend, sind sie oft der erste Schritt eines Angriffs.

8.1 TLS überall

Die API ist ausschließlich über HTTPS erreichbar [68], Klartext-HTTP leitet der Reverse Proxy nur noch um. Caddy erledigt Zertifikate und Umleitung automatisch [73] (Docker-Handbuch, Kapitel 13). Ob die Strecke zwischen Proxy und API ebenfalls TLS braucht, folgt aus der Vertrauensgrenze: Auf demselben kontrollierten Host in einem isolierten Netz kann Klartext vertretbar sein; über Hosts, Clusterknoten, fremdverwaltete Netze oder bei besonders schützenswerten Daten wird sie mit TLS beziehungsweise mTLS geschützt.

8.2 Sicherheits-Header für APIs

Eine API liefert JSON, keine Webseiten, braucht also weniger Header als eine Website. Die folgenden empfiehlt das OWASP *REST Security Cheat Sheet* [68] trotzdem, weil Antworten im Browser landen können:

Header	Wirkung
<code>Strict-Transport-Security: max-age=63072000; includeSubDomains</code>	Browser verwendet für die Domain nur noch HTTPS [74], [75]
<code>Cache-Control: no-store</code>	Antworten mit persönlichen Daten landen nicht in Caches von Browsern oder Proxys [76]
<code>X-Content-Type-Options: nosniff</code>	Browser interpretiert JSON nicht als HTML oder Skript [49]
<code>Content-Security-Policy: frame-ancestors 'none'</code>	Antworten dürfen nicht in fremde Seiten eingebettet werden [77]
<code>Content-Type: application/json</code>	immer korrekt gesetzt, bei FastAPI automatisch

`includeSubDomains` wird nur gesetzt, wenn **jede** Subdomain dauerhaft HTTPS unterstützt; sonst kann HSTS andere Anwendungen unerreichbar machen [74], [75]. `preload` ist eine bewusste, schwer rückgängig zu machende Entscheidung [74] und gehört nicht automatisch in eine Beispielkonfiguration.

Caddyfile

```
api.example.com {
  reverse_proxy api:8000
  request_body {
    max_size 1MB
  }
  header {
    Strict-Transport-Security "max-age=63072000; includeSubDomains"
    X-Content-Type-Options "nosniff"
    Content-Security-Policy "frame-ancestors 'none'"
    Cache-Control "no-store"
  }
  -Server
}
```

```
}
}
```

Das `-Server` [78] entfernt den Header, der Software und Version des Servers verrät. Die Header am Proxy zu setzen hat den Vorteil, dass sie für alle Antworten gelten, auch für Fehlerseiten, die die Anwendung nie erreichen.

8.3 CORS richtig verstehen

Cross-Origin Resource Sharing legt fest, welche fremden Webseiten per JavaScript im Browser auf die API zugreifen dürfen [79]. Zwei Missverständnisse sind verbreitet: CORS schützt die API **nicht** vor Angreifern mit `curl` oder eigenen Programmen, es ist ausschließlich eine Regel für Browser [79]. Und eine zu offene CORS-Konfiguration erlaubt fremden Webseiten, im Namen eingeloggter Benutzer Anfragen zu stellen und die Antworten zu lesen, wenn die API Cookies akzeptiert.

app/main.py

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://notizen.example.com"],      # exakte Liste, kein "*"
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type", "Idempotency-Key"],
    allow_credentials=False,                          # True nur, wenn Cookies nötig sind
    max_age=600,
)
```

ACHTUNG

Die Kombination aus `allow_origins=["*"]` und `allow_credentials=True` [79], [80] oder das Zurückspiegeln des `Origin`-Headers der Anfrage öffnet die API für jede Webseite im Internet. Braucht nur die eigene Web-App Zugriff, genügt genau deren Adresse. Wird die API nur serverseitig (BFF) oder von Apps genutzt, braucht es gar keine CORS-Middleware.

8.4 Cookie-Sessions und CSRF

CORS ist **kein** CSRF-Schutz. Ein BFF setzt sein Session-Cookie mindestens mit `Secure`, `HttpOnly` und möglichst `SameSite=Lax` oder `Strict`; `SameSite=None` ist nur für echte Cross-Site-Anforderungen zulässig und verlangt `Secure` [81]. Zustandsändernde Anfragen (`POST`, `PUT`, `PATCH`, `DELETE`) brauchen zusätzlich ein Synchronizer- oder Double-Submit-CSRF-Token oder eine gleichwertige serverseitige Prüfung von `Origin` und `Fetch-Metadata-Headers` [82]. `SameSite` ist Verteidigung in der Tiefe, kein alleiniger Beweis [82]. Zustandsändernde Aktionen verwenden nie `GET` [82].

8.5 Konfiguration und Geheimnisse

app/settings.py

```
from pydantic import SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="NOTIZEN_", secrets_dir="/run/secrets")
    issuer: str = "https://auth.example.com/application/o/notizen/"  # https://auth.example.com/application/o/notizen/
    audience: str = "https://notizen.example.com/"                  # eigene Audience der API
    jwks_url: str = "https://auth.example.com/.well-known/jwks.json"  # aus Discovery, nicht aus issuer zusammensetzen
    db_password: SecretStr = "password"                               # aus Umgebung oder Datei in /run/secrets
    docs_aktiv: bool = False
    umgebung: str = "produktion"

settings = Settings()  # fehlt ein Pflichtwert: Start schlägt fehl
```

- **Fehlende Pflichtwerte verhindern den Start.** Eine API ohne konfigurierten Issuer darf nicht mit einem Standardwert loslaufen [83].
- `SecretStr` verhindert, dass ein Passwort versehentlich in Logs oder Fehlermeldungen erscheint: Es wird als `*****` ausgegeben, der echte Wert nur mit `.get_secret_value()` [84].
- `secrets_dir` liest Geheimnisse aus Dateien, deren Name dem Namen der Umgebungsvariable entspricht [83], passend zu Docker-Secrets (Docker-Handbuch, Kapitel 12).

8.6 Dokumentation und Debug in Produktion

FastAPI stellt unter `/docs`, `/redoc` und `/openapi.json` automatisch eine vollständige Beschreibung aller Endpunkte bereit [85]. Für eine öffentliche API ist das gewollt; bei einer internen API kann man sie abschalten [85] oder hinter eine Anmeldung legen, um unbeabsichtigte Informationsfreigabe zu reduzieren. Das ist **keine Sicherheitsgrenze**: Jeder Endpunkt bleibt vollständig authentifiziert und autorisiert, auch wenn seine Dokumentation unsichtbar ist.

```
app = FastAPI(
    docs_url="/docs" if settings.docs_aktiv else None,
    redoc_url=None,
    openapi_url="/openapi.json" if settings.docs_aktiv else None,
)
```

Ebenso gilt: kein `debug=True` [86], keine Reload-Server (`fastapi dev`) in Produktion [87], keine Testkonten oder Standardpasswörter, und Fehlermeldungen des Frameworks nicht an Clients durchreichen [66], [68]. Forwarded-Header werden ausschließlich von den explizit konfigurierten Proxy-Netzen akzeptiert [59], [60]; die Anwendung darf nicht gleichzeitig direkt aus dem Internet erreichbar sein.

9 SSRF und fremde APIs

Bei *Server Side Request Forgery* (OWASP API7) bringt ein Angreifer die API dazu, eine von ihm gewählte Adresse aufzurufen [88]. *Unsafe Consumption of APIs* (API10) entsteht, wenn Antworten fremder Dienste zu viel Vertrauen bekommen [89].

9.1 Server Side Request Forgery

Link-Vorschau, Bildimport, frei wählbare Webhooks oder PDF-Erzeugung aus einer URL können Zugriff auf interne Dienste, `localhost`, Docker-Netze oder Cloud-Metadaten [88], [90] ermöglichen.

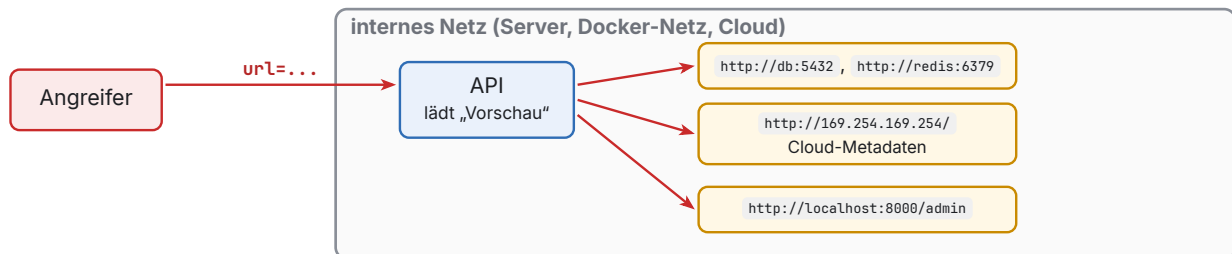


Abb. 5: Die API erreicht Ziele, die von außen nicht erreichbar sind.

Abwehr in Schichten

1. **Keine freien Ziele, wenn vermeidbar:** IDs oder eine exakte Allowlist vertrauenswürdiger Partner statt beliebiger URLs [88], [91].
2. **Strenge URL-Regeln:** nur `https`, kein Benutzername/Passwort, erwarteter Port, normalisierter Host; alle DNS-Ergebnisse auf öffentliche Adressen prüfen [91].
3. **Keine Redirects** [88], [89] – oder jedes Ziel erneut vollständig prüfen.
4. **Antwort begrenzen:** Zeit, Bytes und erlaubte Medientypen, bevor der Body vollständig geladen ist [88], [89].
5. **Egress-Kontrolle:** Proxy/Firewall blockiert interne, Link-Local- und Metadatenziele. Diese Netzwerk-grenze ist bei freien URLs unverzichtbar, weil Anwendungsprüfungen DNS-Rebinding und Parser-Unterschiede nie vollständig ausschließen [91].

app/ssrf.py

```

import ipaddress
import socket
from urllib.parse import urlsplit

import httpx

MAX_BYTES = 100_000

class UnerlaubtesZiel(ValueError): ...

def ziel_vorpruefen(url: str) -> None:
    teile = urlsplit(url)
    if (teile.scheme != "https" or not teile.hostname or
        teile.username is not None or teile.password is not None or
        teile.port not in (None, 443)):
        raise UnerlaubtesZiel("nur https ohne Userinfo auf Port 443")
    adressen = {info[4][0] for info in socket.getaddrinfo(teile.hostname, 443)}
    if not adressen or any(not ipaddress.ip_address(a).is_global for a in adressen):
  
```

```

        raise UnerlaubtesZiel("Zieladresse nicht erlaubt")

def vorschau_laden(url: str) -> str:
    ziel_vorpruefen(url)
    timeout = httpx.Timeout(5.0, connect=2.0)
    with httpx.Client(timeout=timeout, follow_redirects=False) as client:
        with client.stream("GET", url, headers={"User-Agent": "notizen-vorschau"}) as antwort:
            antwort.raise_for_status()
            typ = antwort.headers.get("content-type", "").split(";", 1)[0]
            if typ not in {"text/html", "text/plain"}:
                raise UnerlaubtesZiel("Medientyp nicht erlaubt")
            laenge = antwort.headers.get("content-length")
            if laenge is not None and int(laenge) > MAX_BYTES:
                raise UnerlaubtesZiel("Antwort zu groß")
            daten = bytearray()
            for block in antwort.iter_bytes():
                if len(daten) + len(block) > MAX_BYTES:
                    raise UnerlaubtesZiel("Antwort zu groß")
                daten.extend(block)
            return daten.decode("utf-8", errors="replace")

```

`client.get()` wäre hier falsch: Es lädt den Body vollständig [92], bevor ein nachträgliches `text[:100_000]` greift. Auch der Code oben ist nur eine Anwendungsschicht; zwischen DNS-Prüfung und Verbindung kann sich die Auflösung ändern [91]. Die robuste Produktionslösung verbindet über einen kontrollierten Egress-Proxy beziehungsweise erzwingt die Sperre im Netz.

9.2 Fremde APIs konsumieren

Partnerantworten sind untrusted input: mit Pydantic validieren, Antwortgröße und Zeit begrenzen, TLS nie mit `verify=False` abschalten [89] und pro Partner minimale Zugangsdaten verwenden. Fehler des Partners werden nicht ungefiltert an eigene Clients durchgereicht. Retries erfolgen nur bei sicher wiederholbaren Operationen [93], mit Backoff, Obergrenze und Jitter [94]; Circuit Breaker verhindern Kaskaden [95].

9.3 Webhooks signieren und Replays verhindern

Ein Zeitstempel begrenzt das Replay-Fenster, verhindert aber keine zweite Zustellung **innerhalb** dieses Fensters [96]. Deshalb besitzt jedes Ereignis zusätzlich eine eindeutige ID, die der Empfänger für mindestens die Toleranzdauer atomar speichert und nur einmal akzeptiert [96], [97].

app/webhooks.py

```

import hashlib, hmac, time

def signieren(geheimnis: bytes, koerper: bytes, ereignis: str) -> dict[str, str]:
    zeit = str(int(time.time()))
    inhalt = ereignis.encode() + b"." + zeit.encode() + b"." + koerper
    sig = hmac.new(geheimnis, inhalt, hashlib.sha256).hexdigest()
    return {"X-Ereignis-ID": ereignis, "X-Signatur-Zeit": zeit,
            "X-Signatur-Key": "2026-09", "X-Signatur": sig}

def pruefen(geheimnis, koerper, ereignis, zeit, sig, toleranz=300):
    try:
        sekunden = int(zeit)
    except ValueError:
        return False
    if abs(time.time() - sekunden) > toleranz: return False
    inhalt = ereignis.encode() + b"." + zeit.encode() + b"." + koerper
    erwartet = hmac.new(geheimnis, inhalt, hashlib.sha256).hexdigest()
    return hmac.compare_digest(erwartet, sig)

```

Die ID stammt aus dem persistenten Outbox-Eintrag. Signiert wird der rohe Body (`await request.body()`), nicht neu serialisiertes JSON [96]. Nach gültiger Signatur reserviert der Empfänger die ID per Unique-

Constraint; ein Duplikat bleibt wirkungslos. Die Key-ID ermöglicht Rotation. Rate-/Größenlimits gelten auch hier; Geheimnisse und Rohdaten landen nie im Log.

10 Inventar, Versionen und Lebenszyklus

OWASP API9, *Improper Inventory Management*, beschreibt ein organisatorisches Risiko: APIs, von denen niemand mehr weiß [98]. Eine alte Version `/v1`, die noch läuft, obwohl alle Clients auf `/v2` umgestellt sind und die Sicherheitskorrekturen nur in `/v2` eingebaut wurden [98]. Ein Test-Endpoint aus der Entwicklung. Eine Staging-Instanz mit Kopien echter Daten, aber schwächerem Schutz [98].

10.1 Die OpenAPI-Beschreibung als Inventar

FastAPI erzeugt aus dem Code eine vollständige OpenAPI-Beschreibung [99]. Auch wenn `/openapi.json` in Produktion abgeschaltet ist (Kapitel 8), lässt sie sich im Build erzeugen und versionieren:

```
uv run python -c "import json; from app.main import app; print(json.dumps(app.openapi(),
indent=2))" \
> openapi.json
git diff -- openapi.json          # welche Endpunkte und Schemas änderten sich?
```

Liegt `openapi.json` im Repository, zeigt jeder Pull Request, welche Endpunkte, Parameter und Felder er verändert. Ein OpenAPI-Diff-Werkzeug wie `oasdiff` prüft zusätzlich automatisiert auf brechende Änderungen [100]; ein reines `--stat` zeigt nur Zeilenzahlen. Das macht Änderungen an der Angriffsfläche im Review sichtbar; OWASP empfiehlt, die API-Dokumentation automatisiert in der CI-Pipeline zu erzeugen [98].

10.2 Versionieren und abkündigen

Regel	Umsetzung
Versionsnummer im Pfad	<code>/v1/notizen</code> . Einfach, sichtbar in Logs und leicht am Proxy zu sperren.
Abwärtskompatible Änderungen ohne neue Version	neue optionale Felder, neue Endpunkte
Brechende Änderungen nur in neuer Version	entfernte Felder, geänderte Bedeutung, strengere Validierung
Abkündigung ankündigen	Header <code>Deprecation</code> (RFC 9745) [101] und <code>Sunset</code> (RFC 8594) [102] in jeder Antwort der alten Version
Alte Version wirklich abschalten	zum angekündigten Datum am Proxy sperren und den Code entfernen

```
from fastapi import APIRouter, Depends, Response

def abkuendigung(response: Response) -> None:
    response.headers["Deprecation"] = "@1788220800"          # 1. Sept. 2026 UTC
    response.headers["Sunset"] = "Thu, 31 Dec 2026 23:59:59 GMT"
    response.headers["Link"] = (
        '<https://api.example.com/v2/notizen>; rel="successor-version"'
    )

v1 = APIRouter(prefix="/v1", dependencies=[Depends(abkuendigung)])
```

`APIRouter` besitzt keine Middleware-API; eine Router-Abhängigkeit setzt die Header für alle Version-1-Endpunkte [31]. Alternativ kann `/v1` als eigene FastAPI-Unteranwendung mit Middleware gemountet werden [103]. Der `Link`-Header mit `rel="successor-version"` verweist auf die Nachfolgerversion [104].

10.3 Umgebungen

Staging- und Test-Umgebungen sind häufig schwächer geschützt als Produktion, enthalten aber oft Kopien echter Daten. Deshalb: in Test und Staging nur synthetische oder anonymisierte Daten, eigene Authentik-Provider mit eigenen Schlüsseln pro Umgebung (ein Staging-Token darf in Produktion nie gültig sein, was die Issuer- und Audience-Prüfung aus Kapitel 3 automatisch sicherstellt), und nicht mehr benötigte Umgebungen konsequent abbauen [98].

10.4 Verantwortliches API-Inventar

OpenAPI allein kennt keine Verantwortung. Ein zentrales Inventar hält pro API mindestens Owner und Kontakt, Umgebung und Basis-URL, Datenklassifikation, Authentifizierungsverfahren, externe Erreichbarkeit, Abhängigkeiten, Version, Abkündigungs- und Abschaltdatum fest [98]. CI und Laufzeit-Telemetrie gleichen Soll und Ist ab; unbekannte Hosts, alte Versionen und verwaiste Staging-Systeme werden als Befund behandelt.

11 Sicherheit automatisiert testen

Die Prüfungen aus den vorigen Kapiteln verlieren ihren Wert, wenn eine spätere Änderung sie unbemerkt entfernt. Automatisierte Tests in der CI sorgen dafür, dass jede Änderung erneut gegen die wichtigsten Regeln geprüft wird.



Abb. 6: Von schnellen, statischen Prüfungen zu langsameren Tests gegen die laufende API.

11.1 Statische Analyse und Abhängigkeiten

```

uv run ruff check --select S .           # Sicherheitsregeln (aus flake8-bandit)
uv run pip-audit                         # bekannte Schwachstellen in installierten Paketen
uv run cyclonedx-py environment -o sbom.json
  
```

Die `s`-Regeln von Ruff finden unter anderem hart codierte Passwörter, `verify=False`, unsichere Deserialisierung und `subprocess` mit `shell=True` [105]. `pip-audit` prüft die **installierten** Pakete gegen Datenbanken bekannter Schwachstellen [106]; nach `uv sync --locked` [107] ist das der aufgelöste, gesperrte Stand. Die CycloneDX-SBOM entsteht aus den tatsächlich installierten Paketen und macht diesen Stand für Betrieb und Incident Response nachvollziehbar [108]. Secret-Scanning läuft zusätzlich vor dem Commit und in CI [109]. Ausnahmen werden mit Ticket, Begründung und Ablaufdatum dokumentiert.

11.2 Rechte als Tabelle testen

Der Test aus Kapitel 4 lässt sich zu einer vollständigen Matrix ausbauen: Für jeden Endpunkt wird festgehalten, welche Rolle welchen Statuscode bekommen muss. Neue Endpunkte ohne Eintrag lassen den Test fehlschlagen:

tests/test_rechtematrix.py

```

from fastapi.routing import APIRoute

ERWARTET = {
    ("GET", "/notizen"): {"anonym": 401, "benutzer": 200, "admin": 200},
    ("POST", "/notizen"): {"anonym": 401, "benutzer": 201, "admin": 201},
    ("DELETE", "/admin/benutzer/x/notizen"): {"anonym": 401, "benutzer": 403, "admin": 204},
}

def test_alle_routen_sind_erfasst():
    routen = {(m, r.path) for r in app.routes if isinstance(r, APIRoute)
               for m in r.methods if m != "HEAD" and r.path != "/health"}
    fehlend = routen - set(ERWARTET)
    assert not fehlend, f"Rechte nicht definiert für: {fehlend}"

@pytest.mark.parametrize("methode,pfad,rolle,status", [
    (methode, pfad, rolle, status)
    for (methode, pfad), rechte in ERWARTET.items()
    for rolle, status in rechte.items()
])
def test_rechtematrix(client_fuer, methode, pfad, rolle, status):
  
```

```
antwort = client_fuer(rolle).request(methode, pfad)
assert antwort.status_code == status
```

Die Matrix ist projektspezifisch vollständig zu halten; Platzhalter wie `/x/` werden durch Testdaten ersetzt. Der Ausführungstest ist der eigentliche Wert, die Routenprüfung verhindert nur vergessene Einträge. Zusätzlich gehören negative Tokenfälle in die Suite: `alg=none`, falscher Issuer/Audience, abgelaufen, noch nicht gültig, unbekannte `kid`, fehlende/falsch typisierte Claims und insbesondere ein echtes ID-Token. Warum `alg=none` sowie Issuer und Audience dazugehören, beschreibt RFC 8725 [18].

11.3 Fuzzing aus der OpenAPI-Beschreibung

Schemathesis liest die OpenAPI-Beschreibung und erzeugt daraus automatisch viele Anfragen mit Grenzwerten, falschen Typen, sehr langen Zeichenketten und unerwarteten Kombinationen [110]. Es meldet Serverfehler (5xx), Antworten, die nicht zum Schema passen, und fehlende Validierung (unzulässige Eingaben, die nicht abgelehnt werden) [111]:

```
uv run schemathesis run http://localhost:8000/openapi.json \
  --header "Authorization: Bearer $TEST_TOKEN"
```

Ein 500er bei Schemathesis ist fast immer ein echter Fehler: eine fehlende Längenbegrenzung, ein nicht abgefangener Sonderfall oder ein Datenbankfehler bei ungewöhnlicher Eingabe.

11.4 Dynamischer Scan mit OWASP ZAP

OWASP ZAP bringt einen eigenen Modus für APIs mit, der die OpenAPI-Beschreibung einliest und gegen die gefundenen Endpunkte einen aktiven Scan ausführt [112]:

```
docker run --rm --network notizen_default -v "$PWD:/zap/wrk" \
  zapproxy/zap-stable@sha256:<gepruefter-digest> \
  zap-api-scan.py -t http://api:8000/openapi.json -f openapi -r zap-bericht.html
```

Der Bericht landet als `zap-bericht.html` im aktuellen Verzeichnis, weil dieses nach `/zap/wrk` eingebunden ist [112], [113]. Der Digest wird automatisiert aktualisiert und im Review geprüft; ein beweglicher Tag wie `stable` wäre nicht reproduzierbar, denn Tags können auf ein neues Image zeigen, ein Digest nicht [114]. ZAP findet vor allem Konfigurations- und Injection-Probleme. Autorisierungsfehler wie BOLA brauchen eigene Tests [1]. Aktives DAST läuft nur mit ausdrücklicher Freigabe gegen eine isolierte Umgebung mit synthetischen Daten, nie ungeplant gegen Produktion, weil ein aktiver Scan ein echter Angriff ist [115].

11.5 In der CI

`.gitea/workflows/ci.yml` (Auszug)

```
sicherheit:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@<gepruefter-commit-sha>
    - name: uv installieren
      uses: astral-sh/setup-uv@<gepruefter-commit-sha>
      with:
        version: "<gepruefte-uv-version>" # regelmäßig bewusst aktualisieren
    - run: uv sync --locked
    - run: uv run ruff check --select S .
    - run: uv run pip-audit
    - run: uv run pytest tests/test_rechte.py tests/test_rechtematrix.py
```

Die Platzhalter werden im echten Repository durch geprüfte vollständige Commit-SHAs beziehungsweise Image-Digests ersetzt; die Beispielwerte dürfen nicht unverändert produktiv verwendet werden. Nur ein vollständiger Commit-SHA bindet eine Action unveränderlich [116], [117]. So lädt die CI keinen ungeprüften `curl` | `sh`-Installer und keine unbemerkt verschobenen Actions. Schemathesis und ZAP brauchen eine laufende API mit Datenbank und laufen deshalb in einem getrennten, autorisierten Workflow gegen eine isolierte Testumgebung.

A Checkliste für API-Reviews

AUTHENTIFIZIERUNG

- ☐ Authorization Code + PKCE (`s256`), ggf. Device Code; kein Implicit/Password Grant
- ☐ Redirect-URLs exakt; `state` und bei OIDC `nonce` geprüft
- ☐ Signing Key in Authentik gesetzt (RS256)
- ☐ Signatur, feste Algorithmen, `iss`, API- `aud`, `exp` und Claim-Typen geprüft
- ☐ Access-Token-Vertrag beweisbar; echtes ID-Token im Test abgelehnt
- ☐ Kurze Token-Laufzeit; Refresh-Rotation und `offline_access` bewusst
- ☐ Tokens nie in URLs oder Logs

AUTORISIERUNG

- ☐ Router-weite Anmeldepflicht, öffentliche Endpunkte explizit
- ☐ Besitz-/Freigabeprüfung in jeder Abfrage (BO-LA)
- ☐ Fremde Objekte liefern 404
- ☐ Admin-Funktionen per Gruppe/Scope auf Router-Ebene
- ☐ Alle HTTP-Methoden eines Pfads geprüft
- ☐ Rechtetests mit zwei Benutzern und Rechtenmatrix

DATEN

- ☐ Getrennte Eingabe- und Ausgabemodelle
- ☐ `extra="forbid"` in allen Eingabemodellen
- ☐ Längen-, Wert- und Listengrenzen für jedes Feld
- ☐ `response_model` an jedem Endpunkt
- ☐ Parametrisierte Datenbankabfragen
- ☐ Uploads: Größe, Inhaltsprüfung, Quarantäne/Scan, zufällige Namen

RESSOURCEN

- ☐ Größenlimit am Proxy und in der Anwendung
- ☐ Pflicht-Paginierung mit Höchstwert
- ☐ Rate Limits pro Benutzer und pro IP, strengere für teure Endpunkte
- ☐ `FORWARDED_ALLOW_IPS` nur mit Proxy-Adresse
- ☐ Timeouts für DB und ausgehende Aufrufe
- ☐ Idempotency-Key atomar per Unique-Constraint reserviert

FEHLER UND LOGGING

- ☐ Einheitliche Fehler (RFC 9457), keine Stack-traces
- ☐ Validierungsfehler ohne Eingabewerte
- ☐ Request-ID in Logs und Antworten
- ☐ Keine Tokens, Passwörter oder Bodies im Log
- ☐ Audit-Log für sicherheitsrelevante Aktionen
- ☐ Audit-Log zugriffsbeschränkt, manipulationsgeschützt, mit Löschrifst

KONFIGURATION

- ☐ Nur HTTPS, HSTS und Sicherheits-Header am Proxy
- ☐ CORS mit exakter Origin-Liste oder gar nicht
- ☐ Cookie-Sessions: `Secure`, `HttpOnly`, `SameSite` und eigener CSRF-Schutz
- ☐ Docs/OpenAPI in Produktion aus oder geschützt
- ☐ Pflicht-Konfiguration ohne Standardwerte, `SecretStr`
- ☐ SSRF: Allowlist/IP-Prüfung, Streaming-Limit, keine Redirects, Egress-Sperre
- ☐ Webhooks: HMAC, Zeitstempel, eindeutige Ereignis-ID und Replay-Speicher
- ☐ Alte API-Versionen abgekündigt und abgeschaltet
- ☐ Lockfile, Audit, Secret-Scan, SBOM; CI-Actions und Images unveränderlich gepinnt

B Glossar

Access-Token

Token, das zum Zugriff auf eine API berechtigt. Die einzige Token-Art, die eine API annehmen darf.

Audience (`aud`)

Claim, der angibt, für welchen Empfänger ein Token bestimmt ist.

Authentik

Quelloffener, selbst gehosteter Identity Provider mit OAuth 2.0, OIDC und SAML.

BFF

Backend for Frontend: serverseitiger Teil einer Web-App, der Tokens hält, statt sie dem Browser zu geben.

BOLA

Broken Object Level Authorization: Zugriff auf fremde Objekte über deren ID.

Claim

Einzelne Angabe in einem JWT, z.B. `sub` oder `exp`.

Client Credentials

OAuth-Ablauf, bei dem sich ein Dienst ohne Benutzer mit eigenen Zugangsdaten ein Token holt.

CORS

Browser-Regel, welche fremden Webseiten per JavaScript auf eine API zugreifen dürfen.

CSRF

Cross-Site Request Forgery: eine fremde Seite löst mit automatisch mitgesendeten Cookies eine Aktion aus.

DPoP

Proof-of-Possession-Verfahren nach RFC 9449, das ein Access-Token an einen Client-Schlüssel bindet.

ID-Token

OIDC-Token für den Client mit Angaben zur Anmeldung, nicht für APIs.

Idempotenz

Eigenschaft einer Operation, bei Wiederholung dasselbe Ergebnis zu liefern, ohne doppelte Wirkung.

Issuer (`iss`)

Claim mit der Adresse des Ausstellers eines Tokens.

JWKS

JSON Web Key Set: veröffentlichte öffentliche Schlüssel eines Identity Providers.

JWT

JSON Web Token: signiertes, Base64-kodiertes Token aus Kopf, Nutzlast und Signatur.

Mass Assignment

Übernahme nicht vorgesehener Felder aus einer Anfrage in ein Objekt.

mTLS

Gegenseitiges TLS; kann nach RFC 8705 auch Access-Tokens an ein Client-Zertifikat binden.

OIDC

OpenID Connect: Anmeldeschicht auf OAuth 2.0.

PKCE

Proof Key for Code Exchange: schützt den Authorization Code gegen Abfangen.

Problem Details

Einheitliches Fehlerformat für HTTP-APIs nach RFC 9457.

Rate Limiting

Begrenzung der Anfragen pro Zeitfenster und Aufrufer.

Refresh-Token

Langlebigeres Token, mit dem ein Client neue Access-Tokens holt.

Resource Server

Die API, die Access-Tokens prüft und Daten liefert.

Scope

Benannte Berechtigung, die ein Token enthält, z.B.

`notizen:lesen`.

SSRF

Server Side Request Forgery: die API ruft vom Angreifer gewählte Adressen auf.

Quellen und weiterführende Literatur zu den Begriffen, Standards und Werkzeugen dieses Handbuchs stehen im anschließenden Literaturverzeichnis.

Literaturverzeichnis

Die Nummern verweisen auf die Quellenangaben im Text. Online-Quellen wurden am 1. Oktober 2026 abgerufen. Das Verzeichnis enthält nur Quellen, die im Text zitiert werden.

- [1] OWASP Foundation, „API1:2023 Broken Object Level Authorization“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Oxa1-broken-object-level-authorization/>
- [2] OWASP Foundation, „OWASP Top 10 API Security Risks - 2023“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Ox11-t10>
- [3] OWASP Foundation, „API2:2023 Broken Authentication“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Oxa2-broken-authentication/>
- [4] authentik Security Inc., „Welcome to authentik“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.goauthentik.io/>
- [5] D. Hardt, „The OAuth 2.0 Authorization Framework“, Internet Engineering Task Force (IETF), RFC6749, Okt. 2012. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc6749>
- [6] N. Sakimura, J. Bradley, M. B. Jones, B. de Medeiros, und C. Mortimore, „OpenID Connect Core 1.0 incorporating errata set 2“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://openid.net/specs/openid-connect-core-1_0.html
- [7] T. Lodderstedt, J. Bradley, A. Labunets, und D. Fett, „Best Current Practice for OAuth 2.0 Security“, Internet Engineering Task Force (IETF), RFC9700, Jan. 2025. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc9700>
- [8] A. Parecki, P. De Ryck, und D. Waite, „OAuth 2.0 for Browser-Based Applications“, Internet Engineering Task Force (IETF), RFC10017, Aug. 2026. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc10017>
- [9] W. Denniss, J. Bradley, M. B. Jones, und H. Tschofenig, „OAuth 2.0 Device Authorization Grant“, Internet Engineering Task Force (IETF), RFC8628, Aug. 2019. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc8628>
- [10] authentik Security Inc., „OAuth 2.0 provider“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.goauthentik.io/add-secure-apps/providers/oauth2/>
- [11] N. Sakimura, J. Bradley, und N. Agarwal, „Proof Key for Code Exchange by OAuth Public Clients“, Internet Engineering Task Force (IETF), RFC7636, Sep. 2015. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc7636>
- [12] authentik Security Inc., „Machine-to-machine (M2M) authentication“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://docs.goauthentik.io/add-secure-apps/providers/oauth2/machine_to_machine
- [13] authentik Security Inc., „Release 2024.2“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.goauthentik.io/releases/2024.2/>
- [14] M. B. Jones, J. Bradley, und N. Sakimura, „JSON Web Token (JWT)“, Internet Engineering Task Force (IETF), RFC7519, Mai 2015. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc7519>
- [15] goauthentik contributors, „Add Option to issue RFC9068 compliant access tokens (Issue #22070)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/goauthentik/authentik/issues/22070>
- [16] M. B. Jones, A. Nadalin, B. Campbell, J. Bradley, und C. Mortimore, „OAuth 2.0 Token Exchange“, Internet Engineering Task Force (IETF), RFC8693, Jan. 2020. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc8693>
- [17] authentik Security Inc., „Token exchange“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://docs.goauthentik.io/add-secure-apps/providers/oauth2/token_exchange/
- [18] Y. Sheffer, D. Hardt, und M. B. Jones, „JSON Web Token Best Current Practices“, Internet Engineering Task Force (IETF), RFC8725, Feb. 2020. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc8725>
- [19] V. Bertocci, „JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens“, Internet Engineering Task Force (IETF), RFC9068, Okt. 2021. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc9068>
- [20] J. Richer, „OAuth 2.0 Token Introspection“, Internet Engineering Task Force (IETF), RFC7662, Okt. 2015. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc7662>
- [21] PyJWT contributors, „API Reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pyjwt.readthedocs.io/en/stable/api.html>
- [22] PyJWT contributors, „Usage Examples“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pyjwt.readthedocs.io/en/stable/usage.html>
- [23] FastAPI contributors, „Sub-dependencies“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/dependencies/sub-dependencies/>

- [24] B. Campbell, J. Bradley, N. Sakimura, und T. Lodderstedt, „OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens“, Internet Engineering Task Force (IETF), RFC8705, Feb. 2020. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc8705>
- [25] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. B. Jones, und D. Waite, „OAuth 2.0 Demonstrating Proof of Possession (DPoP)“, Internet Engineering Task Force (IETF), RFC9449, Sep. 2023. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc9449>
- [26] OWASP API Security Project, „API5:2023 Broken Function Level Authorization“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/0xa5-broken-function-level-authorization>
- [27] R. T. Fielding, M. Nottingham, und J. Reschke, „HTTP Semantics“, Internet Engineering Task Force (IETF), RFC9110, Juni 2022. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc9110>
- [28] OWASP Foundation, „Insecure Direct Object Reference Prevention Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
- [29] OWASP Foundation, „Authorization Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- [30] PostgreSQL Global Development Group, „Row Security Policies“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [31] FastAPI, „Bigger Applications - Multiple Files“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/bigger-applications/>
- [32] FastAPI, „Testing Dependencies with Overrides“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/advanced/testing-dependencies/>
- [33] PostgreSQL Global Development Group, „SET“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/sql-set.html>
- [34] OWASP API Security Project, „API3:2023 Broken Object Property Level Authorization“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/0xa3-broken-object-property-level-authorization>
- [35] Pydantic, „Models“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pydantic.dev/docs/validation/latest/concepts/models/>
- [36] FastAPI, „Handling Errors“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/handling-errors/>
- [37] FastAPI, „Response Model - Return Type“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/response-model/>
- [38] OWASP API Security Project, „API4:2023 Unrestricted Resource Consumption“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/0xa4-unrestricted-resource-consumption>
- [39] Pydantic, „Fields (API)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pydantic.dev/docs/validation/latest/api/pydantic/fields/>
- [40] Pydantic, „Network Types“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pydantic.dev/docs/validation/latest/api/pydantic/networks/>
- [41] Pydantic, „Strict Mode“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://pydantic.dev/docs/validation/latest/concepts/strict_mode/
- [42] OWASP Foundation, „SQL Injection Prevention Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [43] SQLAlchemy, „Using SELECT Statements“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://docs.sqlalchemy.org/en/20/tutorial/data_select.html
- [44] OWASP Foundation, „Cross Site Scripting Prevention Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [45] OWASP Foundation, „File Upload Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- [46] SQLAlchemy, „Working with Transactions and the DBAPI“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://docs.sqlalchemy.org/en/20/tutorial/dbapi_transactions.html
- [47] Amazon Web Services, „Download and upload objects with presigned URLs“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>
- [48] MDN Contributors, „Content-Disposition header“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Disposition>
- [49] MDN Contributors, „X-Content-Type-Options header“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options>
- [50] MDN Contributors, „<script> - SVG“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/script>
- [51] OWASP API Security Project, „API6:2023 Unrestricted Access to Sensitive Business Flows“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows>

- [52] Caddy, „request_body (Caddyfile directive)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://caddyserver.com/docs/caddyfile/directives/request_body
- [53] FastAPI, „Path Parameters and Numeric Validations“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/path-params-numeric-validations/>
- [54] PostgreSQL Global Development Group, „Client Connection Defaults“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/runtime-config-client.html>
- [55] M. Nottingham und R. Fielding, „Additional HTTP Status Codes“, Internet Engineering Task Force (IETF), RFC6585, Apr. 2012. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc6585>
- [56] SlowApi, „SlowApi“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://slowapi.readthedocs.io/en/latest/>
- [57] SlowApi, „API reference“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://slowapi.readthedocs.io/en/latest/api/>
- [58] MDN Contributors, „X-Forwarded-For header“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Forwarded-For>
- [59] Uvicorn, „Settings“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://uvicorn.dev/settings/>
- [60] FastAPI, „Behind a Proxy“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/advanced/behind-a-proxy/>
- [61] authentik, „Reputation Policy“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.goauthentik.io/customize/policies/types/reputation/>
- [62] J. Jena und S. Dalal, „The Idempotency-Key HTTP Header Field (Internet-Draft, abgelaufen)“, Internet Engineering Task Force (IETF), draft-ietf-httpapi-idempotency-key-header-07, Okt. 2025. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>
- [63] PostgreSQL Global Development Group, „INSERT“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/sql-insert.html>
- [64] PostgreSQL Global Development Group, „Index Uniqueness Checks“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/index-unique-checks.html>
- [65] C. Richardson, „Pattern: Transactional outbox“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://microservices.io/patterns/data/transactional-outbox.html>
- [66] OWASP Foundation, „API8:2023 Security Misconfiguration“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Oxa8-security-misconfiguration>
- [67] M. Nottingham, E. Wilde, und S. Dalal, „Problem Details for HTTP APIs“, Internet Engineering Task Force (IETF), RFC9457, 2023. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc9457>
- [68] OWASP Foundation, „REST Security Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
- [69] OWASP Foundation, „Logging Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- [70] S. Kanzhelev, M. McLean, A. Reitbauer, B. Drutu, N. Molnar, und Y. Shkuro, „Trace Context“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.w3.org/TR/trace-context/>
- [71] OpenTelemetry Authors, „Context propagation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://opentelemetry.io/docs/concepts/context-propagation/>
- [72] European Parliament and Council of the European Union, „Regulation (EU) 2016/679 (General Data Protection Regulation), Art. 5: Principles relating to processing of personal data“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gdpr-info.eu/art-5-gdpr/>
- [73] Caddy, „Automatic HTTPS“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://caddyserver.com/docs/automatic-https>
- [74] MDN contributors, „Strict-Transport-Security“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security>
- [75] J. Hodges, C. Jackson, und A. Barth, „HTTP Strict Transport Security (HSTS)“, Internet Engineering Task Force (IETF), RFC6797, Nov. 2012. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc6797>
- [76] MDN contributors, „Cache-Control“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cache-Control>
- [77] MDN contributors, „Content-Security-Policy: frame-ancestors“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/frame-ancestors>
- [78] Caddy, „header (Caddyfile Directive)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://caddyserver.com/docs/caddyfile/directives/header>
- [79] MDN contributors, „Cross-Origin Resource Sharing (CORS)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- [80] FastAPI, „CORS (Cross-Origin Resource Sharing)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/cors/>
- [81] MDN contributors, „Set-Cookie“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

- [82] OWASP Foundation, „Cross-Site Request Forgery Prevention Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- [83] Pydantic, „Settings Management“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://pydantic.dev/docs/validation/latest/concepts/pydantic_settings/
- [84] Pydantic, „Pydantic Types“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pydantic.dev/docs/validation/latest/api/pydantic/types/>
- [85] FastAPI, „Metadata and Docs URLs“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/tutorial/metadata/>
- [86] Starlette, „Exceptions“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.starlette.dev/exceptions/>
- [87] FastAPI, „FastAPI CLI“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/fastapi-cli/>
- [88] OWASP Foundation, „API7:2023 Server Side Request Forgery“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Oxa7-server-side-request-forgery>
- [89] OWASP Foundation, „API10:2023 Unsafe Consumption of APIs“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Oxaa-unsafe-consumption-of-apis>
- [90] Amazon Web Services, „Access instance metadata for an EC2 instance“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instancedata-data-retrieval.html>
- [91] OWASP Foundation, „Server-Side Request Forgery Prevention Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- [92] Encode, „QuickStart“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.python-httpx.org/quickstart/>
- [93] Amazon Web Services, „Retry with backoff pattern“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/retry-backoff.html>
- [94] M. Brooker, „Exponential Backoff And Jitter“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>
- [95] M. Fowler, „Circuit Breaker“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://martinfowler.com/bliki/CircuitBreaker.html>
- [96] Stripe, „Registrieren Sie Stripe-Ereignisse in Ihrem Webhook-Endpoint“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.stripe.com/webhooks>
- [97] GitHub, „Best practices for using webhooks“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/webhooks/using-webhooks/best-practices-for-using-webhooks>
- [98] OWASP Foundation, „API9:2023 Improper Inventory Management“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://api-security.owasp.org/editions/2023/en/Oxa9-improper-inventory-management>
- [99] Ramírez, Sebastián, „Extending OpenAPI“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/how-to/extending-openapi/>
- [100] oasdiff, „oasdiff“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.oasdiff.com/>
- [101] S. Dalal und E. Wilde, „The Deprecation HTTP Response Header Field“, Internet Engineering Task Force (IETF), RFC9745, 2025. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc9745>
- [102] E. Wilde, „The Sunset HTTP Header Field“, Internet Engineering Task Force (IETF), RFC8594, Mai 2019. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc8594>
- [103] Ramírez, Sebastián, „Sub Applications - Mounts“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/advanced/sub-applications/>
- [104] A. Brown, G. Clemm, und J. Reschke, „Link Relation Types for Simple Version Navigation between Web Resources“, Internet Engineering Task Force (IETF), RFC5829, Apr. 2010. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc5829>
- [105] Astral, „Rules“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/ruff/rules/>
- [106] Python Packaging Authority, „pip-audit“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pypi.org/project/pip-audit/>
- [107] Astral, „Locking and syncing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/uv/concepts/projects/sync/>
- [108] CycloneDX, „Usage“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://cyclonedx-bom-tool.readthedocs.io/en/latest/usage.html>
- [109] OWASP Foundation, „Secrets Management Cheat Sheet“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- [110] Schemathesis, „Schemathesis Documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://schemathesis.readthedocs.io/en/stable/>
- [111] Schemathesis, „Command-Line Interface“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://schemathesis.readthedocs.io/en/stable/reference/cli/>
- [112] ZAP, „ZAP API Scan“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.zaproxy.org/docs/docker/api-scan/>

- [113] ZAP, „ZAP Docker“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.zaproxy.org/docs/docker/about/>
- [114] Docker Inc., „Building best practices“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.docker.com/build/building/best-practices/>
- [115] ZAP, „Getting Started with ZAP“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.zaproxy.org/getting-started/>
- [116] GitHub, „Secure use reference“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/actions/reference/security/secure-use>
- [117] Astral, „setup-uv“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/astral-sh/setup-uv>